



## Full length article

## Combustion chemistry acceleration with DeepONets

Anuj Kumar, Tarek Echekki \*

Department of Mechanical and Aerospace Engineering, North Carolina State University, Campus Box 7910, Raleigh, 27695, NC, USA

## ARTICLE INFO

## Keywords:

Operator learning  
Machine learning  
Stiff chemistry integration  
Surrogate modeling  
Latent space kinetics identification

## ABSTRACT

A combustion chemistry acceleration scheme for implementation in reacting flow simulations is developed based on deep operator nets (DeepONets). The scheme is based on a mapping of a subset of the thermochemical scalars' vector between incremental and adaptive time steps. The subset corresponds to variables that are adequate to represent the evolution of the chemical system. The DeepONet-based scheme also relies on a mapping of the solution of this subset onto a corresponding solution at an adaptive time increment to overcome the restrictive requirements of integrating such solutions with stiff chemistry. Training for the DeepONet is also implemented differently from previous approaches to the solution of PDEs with DeepONets. Instead of constructing solutions from their initial to their final states using DeepONets, the training relies on prescribed short-horizon time windows for training where intermediate solutions also serve as initial states for training. An additional framework of latent-space kinetics identification with modified DeepONet is proposed, which enhances the computational efficiency and widens the applicability of the proposed scheme. The scheme is demonstrated on the "simple" chemical kinetics of hydrogen oxidation and the more complex chemical kinetics of n-dodecane high- and low-temperatures. The proposed framework accurately learns the chemical kinetics and efficiently reproduces species and temperature temporal profiles. Moreover, a very large speed-up with a good extrapolation capability is also observed with the proposed scheme.

## 1. Introduction

Chemistry integration is the principal bottleneck in combustion simulations. Different approaches have been implemented to accelerate chemistry. They include techniques of chemistry tabulation, such as the *in situ* adaptive tabulation (ISAT) [1], and the piecewise reusable implementation of solution mapping (PRISM) [2], adaptive chemistry, such as dynamic approaches (see for example Refs. [3–6]), manifold-based methods, such as intrinsic low-dimensional manifolds (ILDLM) [7] and computational singular perturbation (CSP) methods [8]. Machine learning (ML) techniques were also implemented for combustion chemistry that adopted strategies to represent reaction rates using artificial neural networks (ANNs) ([9–22].)

In recent years, additional approaches have been proposed to accelerate chemistry integration in combustion using machine learning (ML) tools. They include the studies by Owoyele and Pal [23], Zhang et al. [24], Kim et al. [25] and Galassi et al. [26]. The ChemNODE approach proposed by Owoyele and Pal [23] combines a stiff ODE solver and a deep neural network that minimizes the solution of chemistry integration based on the law of mass action and the ANN-based reaction rates. Zhang et al. [24] trained a deep neural network (DNN) to project a solution of the thermochemical state vector between two time steps. Kim et al. [25] proposed a method for learning neural ODEs

for reacting systems and proposed strategies to overcome the chemistry stiffness. Galassi et al. [26] proposed the use of ANN regression as a more efficient surrogate for the local projection basis within the computational singular perturbation (CSP) approach [8], thus lessening the bottleneck within the CSP chemistry acceleration technique. Other implementations of ML-based methods include approaches to evaluate rates of reactions.

Among the ML-based complex networks proposed in recent years, deep operator networks, or DeepONets, [27] have emerged as effective networks to learn continuous nonlinear operators mapping functions between infinite-dimensional spaces based on the universal approximation theorem for operators [28]. The method has been implemented in different applications [29–34], including combustion [35–40].

The work of Venturi and Casey [35] is of particular relevance to the present study and represents a starting point for this work. In their recent study, they implemented the DeepONet to predict the time evolution of temperature and the species during the oxidation of hydrogen in air at high temperatures. They proposed an enhancement to the DeepONet called flexDeepONet, which adds a pre-transformation network designed to generate a moving reference frame that accommodates solutions with self-similar profiles associated with combustion

\* Corresponding author.

E-mail address: [techekk@ncsu.edu](mailto:techekk@ncsu.edu) (T. Echekki).

chemistry. They showed that their proposed flexDeepONet outperforms the standard DeepONet [27] in terms of accuracy and model size. The number of trainable parameters in their flexDeepONet network is approximately 95% less than those of the standard DeepONet. A similar pre-transformation network was proposed by Hadorn et al. [41] to address nonlinear functions with sharp features.

In this study, we are interested in combustion chemistry integration and the potential of using DeepONet to accelerate it. In addition to the earlier work of Venturi and Casey [35], an independent implementation of DeepONet for combustion chemistry integration has been developed by Goswami et al. [40] (also see Kumar and Echehki [37] and Gosmawi et al. [38] for earlier versions of the two approaches). Comparisons of the implementation between our present approach and the work of Goswami et al. are presented in the Methodology Section 2.

We are particularly interested in implementing our approach for the oxidation of complex hydrocarbon fuels. For these fuels, the so-called “curse of dimensionality” imposes stringent constraints on any alternative approaches to integrate or predict their chemistry. For this problem, it is important to recognize several key attributes of combustion chemistry, which must be addressed or exploited with DeepONet.

The first attribute is associated with the multiscale nature of chemical kinetics. Practical combustion reactions involve hundreds and even thousands of species whose chemistry can be represented by an even larger number of elementary reactions. There is a wide range of scales that characterize this chemistry, hence the inherent stiffness of chemical systems. More importantly, the interactions of chemical reactions result in complex features of species’ temporal or spatial profiles.

Second, chemistry acceleration can exploit the presence of low-dimensional manifolds in chemical systems. These manifolds are exhibited in the presence of a subset of the species or a linear or nonlinear basis that incorporates the inherent correlation and cause-and-effect relations driving the evolution of chemistry. They are also exhibited in the chemical time scales, which have motivated the development of methods such as intrinsic low-dimensional manifolds (ILDM) [7] and computational singular perturbation (CSP) methods [8].

Finally, features in the evolution of species’ temporal or spatial profiles can be exploited to reduce the complexity of modeling chemical systems. For example, during the low-temperature chemistry of complex hydrocarbons, the oxidation process may be characterized by a two-stage ignition process. The mixture conditions may dictate the timing and intensity of these 2 stages; yet, the presence of common features may help in modeling them. The Shift-DeepONet by Hadorn et al. [41] and the flexDeepONet by Venturi and Casey [35] represent recent variants of the DeepONet that exploit the occurrence of such features. Both approaches involve the addition of a sub-network that allows the automatic discovery of self-similar profiles in the solution.

The objective of this study is to develop an efficient combustion chemistry acceleration approach based on DeepONet called React-DeepONet. The method exploits the inherent advantages of DeepONets and their variants to build an efficient and robust implementation that can be applied to complex fuels. As a strategy, we propose the implementation of a DeepONet that maps a solution of a given vector of thermochemical scalars onto a new vector at a later adaptive time increment. The proposed DeepONet features a novel training mechanism that augments the amount of data for training and an augmentation of the network architecture through the use of a parameter input sub-network called ParamNet. The method is demonstrated for a simple fuel,  $H_2$ , and a more complex fuel, n-dodecane, that exhibits different dominant reactions during its low- (LTO) and high-temperature (HTO) oxidations. The method is discussed in Section 2. Then, results are presented in Section 3. Finally, conclusions and discussions of future work are presented in Section 4.

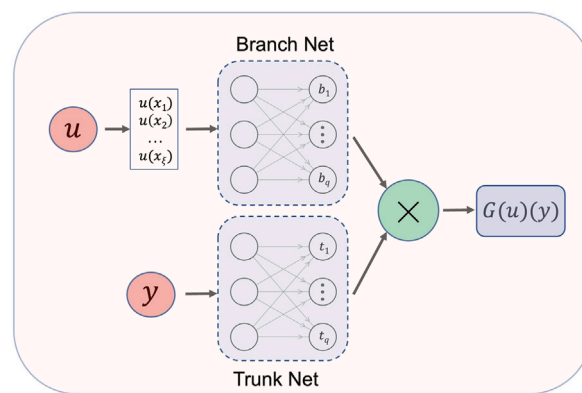


Fig. 1. Architecture of the basic DeepONet by Lu et al. [27].

## 2. Methodology

As discussed earlier, DeepONets represent relatively complex ML-based networks with potential for different applications, including the integration of combustion chemistry. Fig. 1 illustrates the architecture of the DeepONet based on the original work of Lu et al. [27]. It uses a neural network architecture inspired by the universal approximation theorem of Chen and Chen [17] with two coupled sub-networks to learn an operator,  $G$ . The operator takes an input function  $u$  and results in an output function  $G(u)$ . The output function,  $G(u)$ , is evaluated at any independent variable,  $y$ , and results in a real number,  $G(u)(y)$ . Within our proposed approach,  $u$  represents a solution vector of thermochemical scalars at a prescribed time,  $y$  represents a prescribed time increment, and the output function,  $G(u)$  represents a predicted solution for this thermochemical scalars’ vector at the prescribed time increment from the input solution,  $u$ .

As shown in Fig. 1, the basic input of the DeepONet network of Lu et al. [27] consists of two separate components  $[u(x_1), u(x_2), \dots, u(x_\xi)]^T$  and  $y$  and both inputs are handled separately by two sub-networks. The first sub-network is the “Trunk net”, which takes the variable,  $y$ , as an input and outputs  $(t_1, t_2, \dots, t_p)$ , indicating  $p$  neurons in its output. The second sub-network, the “Branch” net, takes input  $[u(x_1), u(x_2), \dots, u(x_\xi)]^T$  at fixed sensor points,  $\xi$ , and outputs  $(b_1, b_2, \dots, b_p)$ , with again  $p$  neurons at its output. The final output of the DeepONet is obtained by merging the outputs of the Trunk and the Branch networks via a dot product.

$$G(u)(y) \approx \sum_{k=1}^p \underbrace{b_k(u(x_1), u(x_2), \dots, u(x_\xi))}_{\text{Branch}} \underbrace{t_k(y)}_{\text{Trunk}} \quad (1)$$

Within the context of our present modeling framework, we are only considering a single sensor location in the form of time. Therefore,  $\xi = 1$ . The DeepONet guarantees the universal approximation of an operator by its construction and it has been shown to accurately approximate various implicit and explicit operators [27,42,43]. Recent studies on DeepONet provided theoretical upper bounds for the approximation error in terms of the network size and the operator type [44,45]. As a supervised ML technique, training is implemented on the function values for  $u$  at different instances of  $y$ .

The proposed React-DeepONet borrows some important features of the standard DeepONet of Lu et al. [27] and, like the recent work of Venturi and Casey [35], accommodates some of the key features discussed above related to combustion chemistry. However, it also includes important variations designed specifically to implement DeepONet for complex fuels while maintaining computational efficiency and reducing storage requirements in reacting flow simulations. These variations are discussed below and include:

- We adopt a solution mapping approach that is designed to predict the solution of the chemical system, at small and adaptive time increments. This React-DeepONet is tasked with the prediction of a subset of the thermochemical scalars, called representative scalars [46], from which the remaining species can be recovered.
- The training of these solutions is also implemented on time windows to accommodate the large variations of the representative species profiles over time. An important consequence of this DeepONet architecture and training procedure is that training data is augmented by considering every intermediate solution as an initial condition and different time steps within the time window as input parameters to the network.
- An additional input sub-network is adopted to enable the addition of additional parameters to the solution profiles. Such parameters may include variations in pressure, equivalence ratio, dilution rates, or other key parameters that can additionally prescribe the initial conditions.
- A latent space React-DeepONet, or LS-React-DeepONet, is also proposed. In this architecture, a dimensionality reduction on the representative scalars is carried out using an encoder within an autoencoder architecture to yield a reduced set of variables in latent space. React-DeepONet is implemented on these variables to predict their solution over a small time step. Then, a decoder is used to recover the original representative scalars. The LS-React-DeepONet is designed to reduce the size of the DeepONet architecture, especially for complex fuels.

### 2.1. Overall architecture of React-DeepONet

In this study, we illustrate the React-DeepONet using a 0D, homogeneous, constant-pressure reactor model. Within this model temperature and species equations are evolved based on a system of ODEs, which are solved as an initial value problem taking the form:

$$\frac{d\phi}{dt} = S(\phi, p_0) \quad (2)$$

$$\phi(0) = \phi_0 \quad (3)$$

where  $\phi$  is the thermochemical composition vector, which includes species mass fractions and temperature. The term,  $S$  is a function of thermochemical scalars,  $\phi$ , and represents the chemical reaction source terms based on the law of mass action. The reaction system is stiff and computationally expensive to solve since all the reactions temporally evolve on different time scales and involve nonlinear evaluations of multiple chemical reactions.

Both this study and the study of Venturi and Casey [35] adopted this model problem to construct a solution for the thermochemical scalars as a function of time. For comparison between the two approaches, we illustrate their DeepONets without the added variations (i.e. flexDeepONet, SVD-DeepONet, and the specific implementation of the Shift/Stretch-Net), which are discussed below.

Fig. 2 compares the “basic” architectures of the DeepONet formulation of Venturi and Casey [35] and our approach, which mimic the standard DeepONet, which do not feature measures to exploit self-similar profiles in the thermochemical scalars’ solutions. The figure is used to illustrate the difference in input and output channels associated with both DeepONets.

In the Venturi and Casey [35] approach the inputs correspond to the initial solution vector,  $\phi(0)$  (on the Branch Net side), and the time variable,  $t$ , (on the Trunk Net). Therefore, their approach is based on predicting with DeepONet the entire solution vector as a function of time,  $\phi(t)$ , over a prescribed integration time. This solution corresponds to the output of the DeepONet.

In contrast, in our approach, the inputs correspond to the solution vector at a given time,  $\phi(t)$ , and a prescribed time increment,  $\delta t$ , to map this solution vector into an output,  $\phi(t + \delta t)$ . This choice of inputs

and outputs alters completely the DeepONet formulation in terms of its scope and training approaches.

First, the motivation for our approach is based on the coupling of DeepONet with combustion solvers for reacting flows where direct chemistry integration in the reaction source operator is replaced by a DeepONet projection at a prescribed time step. This integration is a bottleneck in simulations of reacting flows requiring an adequate model for combustion chemistry.

Second, there are important consequences to how the DeepONets are trained even given the same set of data. Here, we will expand on these consequences for our approach to DeepONet. In our approach, every output stored for the solution vector from the simulations at a given time step in the training data serves as an input to the DeepONet. Solutions at subsequent time steps,  $\phi(t + \delta t)$ , up to a target window,  $\Delta t$ , can serve as outputs for training the DeepONet.

This procedure is illustrated in Fig. 2(b). A 1D profile of one component of the solution vector is specified as a function of time. A numerical solution generates discrete outputs of the thermochemical scalars at small increments of time, and these outputs correspond to the training data. A section of this 1D profile shown in the figure illustrates a span of this output data starting with  $\phi(t)$  and ending with  $\phi(t + \Delta t)$ . Multiple solution outputs are generated within that window that corresponds to much smaller increments of  $\Delta t$ . The solution at  $t$  can serve as input to the Branch Net, while different values for the time increments,  $\delta t$ , can be prescribed with values ranging from 0 to  $\Delta t$ . These time increments correspond to the input of the Trunk Net with the corresponding output of the DeepONet of  $\phi(t + \delta t)$ .

The adopted projection approach across small time increments with a prescribed  $\Delta t$  also accommodates the strong dominant chemistry variations during fuel oxidation. This is true for complex hydrocarbon fuels and may be more pronounced during their low-temperature oxidation. Adopting a short horizon for training (training over a  $\Delta t$  compared to the prediction of the entire solution profiles) is designed to enhance its robustness.

Having shown the main differences in the inputs and outputs of the DeepONets proposed by Venturi and Casey [35] and the present work, we now discuss the architecture of the React-DeepONet proposed here, which is shown in Fig. 3. Similar to the standard DeepONet, it features a Branch Net and a Trunk Net. As stated earlier, the inputs feature the solution vector  $\phi(t)$  at a prescribed time  $t$  and the time increment,  $\delta t$ . The solution vector may be prescribed by pressure, temperature, composition, or any combination that dictates the state of this mixture. The two inputs are handled separately by two sub-networks: (1) the Branch Network for the solution vector  $\phi(t)$  and (2) the Trunk Network for the parameter  $\delta t$ . The output corresponds to the solution vector  $\phi(t + \delta t)$  at a later increment in time, at  $t + \delta t$ . This output is obtained by merging the outputs of the branch and Trunk Networks via a dot product, as discussed earlier.

Similar to the work of Venturi and Casey [35], we adopt a pre-transformation network, the ShiftNet. This sub-network is designed to determine an optimum shift,  $\Delta t$ , in log scale which aligns different profiles of the training data. The ShiftNet stretches the time axis in linear scale (or shifts it in log scale) and finds a reference time for each profile such that the maximum gradients of the profiles are aligned. Note that the flexDeepONet in the work of Venturi and Casey [35] can also serve to scale the solution profiles. This scaling is not implemented in our present ShiftNet since we found that establishing self-similarities in temporal profiles is more crucial, and can be more efficient, than implementing shifting and scaling optimizations.

Within the React-DeepONet architecture, the ShiftNet is a fully connected network with an input corresponding to the solution vector and an output that corresponds to the solution shift,  $\Delta t$ . This shift then augments the prescribed time increment to serve as the input to the Trunk Net. During the prediction phase of the React-DeepONet, the trained ShiftNet network with its determined parameters predicts the proper shift,  $\Delta t$ , for a given solution vector. The weights of the

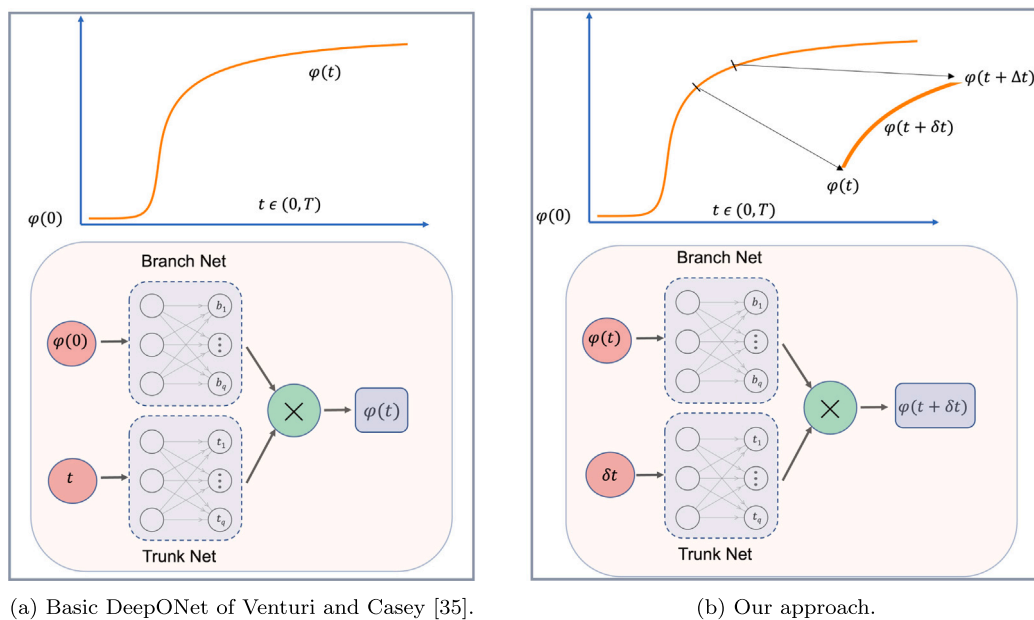


Fig. 2. Inputs and outputs of the basic form of Venturi and Casey and our approach.

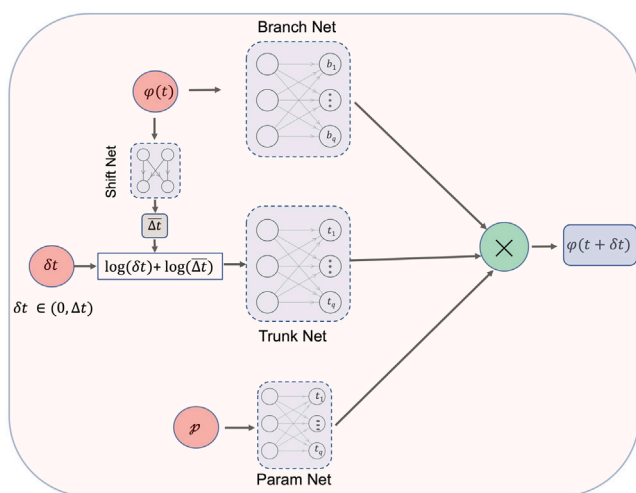


Fig. 3. React-DeepONet architecture.

ShiftNet are determined as part of the training/optimization of the React-DeepONet to minimize its loss function. As illustrated in Fig. 3, the shift  $\Delta t$  is incorporated as a sum:  $\log \delta t + \log \Delta t$ , which is used as an input to the Trunk Net.

In addition to the ShiftNet, we also implement a ParamNet. The outputs, from the ParamNet, the Branch Net, and the Trunk Net are concatenated to produce a single output vector. This sub-network provides an additional input network to accommodate additional parameters of the solution. For example, different equivalence ratios, mixture dilution rates, or pressures can be implemented as part of the ParamNet inputs to accommodate the variations in the solution profiles resulting from these parameters. The need for a ParamNet is largely dictated by the extent of these variations and the scope of the training data for a given problem.

It is important to note that, in addition to mapping solutions across time increments, the current React-DeepONet is also able to predict chemical source terms,  $S$ , of the modeled species,  $\phi$ , based on the

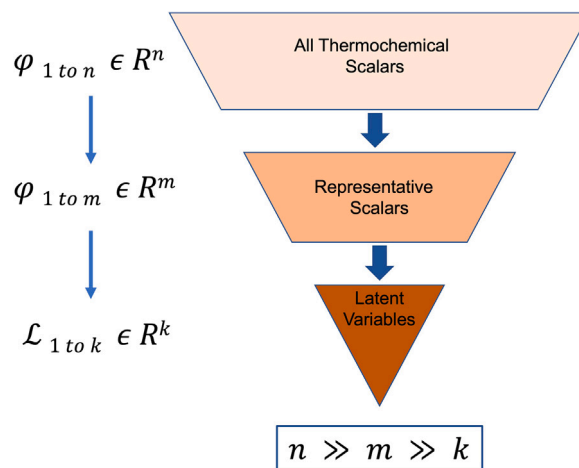


Fig. 4. Thermochemical scalars' hierarchy.

evaluation of the gradients of the predicted solutions vs. time.

$$S(\phi) = \frac{\partial \phi}{\partial(\delta t)} = \frac{\partial \phi}{\partial t} \quad (4)$$

Another important distinction of the present study is the use of a modified fully connected neural network [47] for all the sub-networks and the use of a single DeepONet for all species. The single DeepONet is configured to output all the thermochemical scalars in the subset by equally dividing the final dot layer, where each segment specifically caters to each respective scalar [48]. Accordingly, the input to the Branch Net and the output predict the evolution of these species over a small increment of time. In the recent work of Venturi and Casey [35], each species is predicted by its own DeepONet. The use of a single DeepONet, however, must address the inherent cost of predicting multiple species all at once. In this study, we use two dimensionality reduction approaches to represent the evolution of the chemical system in terms of a subset of species or latent space variables.

Fig. 4 illustrates the hierarchy of scalars that can result in a significant reduction of the modeled scalars in DeepONet. At the top of the hierarchy, all thermochemical scalars, including temperature

and species in the chemical mechanism yield a solution vector of dimension  $n$ . For complex fuels,  $n$  can be of the order of hundreds and even thousands of thermochemical scalars. By the selection of a subset of the scalars using representative scalars, the number can be reduced to  $m$ . The representative scalars,  $(\phi_1, \phi_2, \dots, \phi_m) \in \mathbb{R}^m$ , include temperature, major species and intermediate species that play a key role in the oxidation process and with which the non-representative species,  $(\phi_{m+1}, \phi_{m+2}, \dots, \phi_n) \in \mathbb{R}^{n-m}$ , are highly correlated. Owing to the strong correlation, the remaining scalars  $(\phi_{m+1}, \phi_{m+2}, \dots, \phi_n)$  are recovered through regression in terms of the representative scalars [49,50]. In earlier studies, we have trained ANNs to recover these remaining scalars [50,51]. The number of representative scalars,  $m$ , can reduce the number of required scalars in the chemical system to an order of 10.

A further reduction in the dimensionality of the system can be achieved through a latent space, as illustrated in Fig. 4. In this study, we implement autoencoders to discover the latent space variables from the representative scalars subset of the solution vector.

## 2.2. Reduction of the solution vector of thermochemical state using representative scalars

Another important variation of the present approach compared to the approach of Venturi and Casey [35] is the solution of a subset of the thermochemical scalars instead of all scalars representing the chemical system. In this section, we discuss a dimensionality reduction method in which React-DeepONet is implemented on a selected subset of scalars, denoted as representative scalars [46]. The representative scalars correspond to representative species and temperature from which other species can be recovered [46]. In the study of Alqahtani and Echehki [20], these representative scalars are determined using principal component analysis (PCA) implemented on reaction rates from simulation data. The procedure determines the species whose reactions are crucial to other remaining species. In the present study, we use the same PCA procedure as in Alqahtani and Echehki [20] to determine the representative scalars. The procedure for selecting the representative species, along with temperature representative scalars, is briefly summarized below [20]:

- Starting with simulation data, which is also used for training React-DeepONet, each species' reaction rate,  $S(\phi)$ , is centered by subtracting their mean over the entire data range:

$$S_k^* = S_k - \bar{S}_k \quad (5)$$

where  $S_k$  is the  $k$ th species reaction rate and  $\bar{S}_k$  is its average

- The covariance matrix is constructed from the centered reaction rates. The component of this matrix can be expressed as:

$$C_{ij} = \frac{1}{n-1} \sum_{k=1}^{n-1} S_i^{*k} S_j^{*k}. \quad (6)$$

The matrix has dimensions of  $(n-1) \times (n-1)$ , where  $n-1$  is the number of species in the chemical system. The full set of thermochemical scalars also includes temperature.

- A first PCA is implemented on this covariance matrix, which constructs an  $(n-1) \times (n-1)$  matrix of  $(n-1)$  eigenvectors. These eigenvectors are listed in descending order of the magnitude of the eigenvalues and their contribution to the total data variance. The resulting principal components (PCs) can be expressed in terms of the original centered reaction rates as follows:

$$\Psi = \mathbf{Q}^T \mathbf{S}^* \quad (7)$$

where  $\mathbf{Q}^T$  corresponds the matrix of eigenvectors. The coefficients of the eigenvectors indicate the individual contributions of the centered reaction rates to the PCs:

$$\Psi_j = q_{1,j} S_1^* + q_{2,j} S_2^* + \dots + q_{(n-1),j} S_{(n-1)}^* \quad (8)$$

where the  $q_{i,j}$ 's are the coefficient of matrix  $\mathbf{Q}^T$  corresponding to the  $i$ th species source term and the  $j$ th species. The first few PCs represent the bulk of the variance of  $\mathbf{S}^*$ .

- Since the first PCs represent the bulk of the variance in the solution vector  $\mathbf{S}^*$ , we only retain the first  $N_{PC}$  PCs, which contain 99% of this variance. The retained PCs are expressed as:

$$\Psi_{\text{red}} = \mathbf{A}^T \mathbf{S}^* \quad (9)$$

where  $\mathbf{A}^T$  includes the leading  $N_{PC}$  eigenvectors of  $\mathbf{Q}^T$ .

- Based on this reduction, we can re-write a similar expression as in Eq. (8) and order the sum in descending order from highest to lowest magnitudes of  $q_{i,j}$ , thus ordering the species with the greater coefficients to the PCs.
- We implement a cut-off criterion on the re-ordered sum in Eq. (8),

$$\frac{\sum_{i=1}^b \|q_{i,j}\|}{\sum_{i=1}^{n-1} \|q_{i,j}\|} \times 100 > \text{cut-off } \%, j = 1, \dots, N_{PC} \quad (10)$$

- Similar to the study of Alqahtani and Echehki [20], the cut-off criterion is set at 99% from which a  $b$  number of species are retained from the original  $n-1$  species.
- The same procedure as the previous steps is, then, implemented on the retained  $b$  species to reduce the original vector of thermochemical scalars from  $n$  to  $m$ . The two-step PCA is designed to iteratively reduce the number of retained species using the hierarchy established by the contribution of their reactions.

It is important to note that other methods for determining representative scalars can be adopted. They include the use of directed relations graphs (DRG) [52], a popular tool for chemistry reduction, the global pathways selection (GPS) algorithm of Gao et al. [53] or the manifold-informed representative species selection method by Zdybal et al. [54].

## 2.3. Latent space kinetics identification: The LS-React-DeepONet

As already discussed, the presence of an even lower dimensional manifold for the representative scalars can be exploited to accelerate the training and the implementation of the DeepONet frameworks for chemistry acceleration in combustion. The dimensionality of the representative dataset can be further reduced with various dimensionality reduction techniques without incurring any discernable loss in its representation capability. Recently, Oommen et al. [55] proposed a framework that integrates a convolutional autoencoder with DeepONet to model the dynamic evolution of a two-phase mixture. Oomen et al. [55] modify the DeepONet architecture by replacing the Branch Net with a pre-trained convolutional autoencoder such that inputs of the network are in the original space of an image.

As an extension to the React-DeepONet for representative scales, and in contrast to the DeepONet framework of Oommen et al. [55] which take input in the original space of images, we propose a strategy to accelerate chemistry integration by training and predicting the DeepONet in latent space. Within the LS-React-DeepONet, autoencoders are proposed, which have been shown to effectively reduce the dimensionality and identify the latent space [51,56] in chemical systems. Autoencoders provide a nonlinear mapping from a high dimensional space (of dimension  $m$ ) to a low dimensional space (of dimension  $k$ ) [57,58], as illustrated in Fig. 4.

The autoencoder is trained on the simulation data. The input to the autoencoder corresponds to the vector of representative scalars from the simulation data. In the output, the autoencoder attempts to reproduce the same vector values. The encoder portion provides a model for converting this initial vector into latent space variables. While the decoder reconstructs the original representative scalars from these variables. Once the autoencoder is trained, the simulation data is converted to a lower-dimensional vector of the latent variables

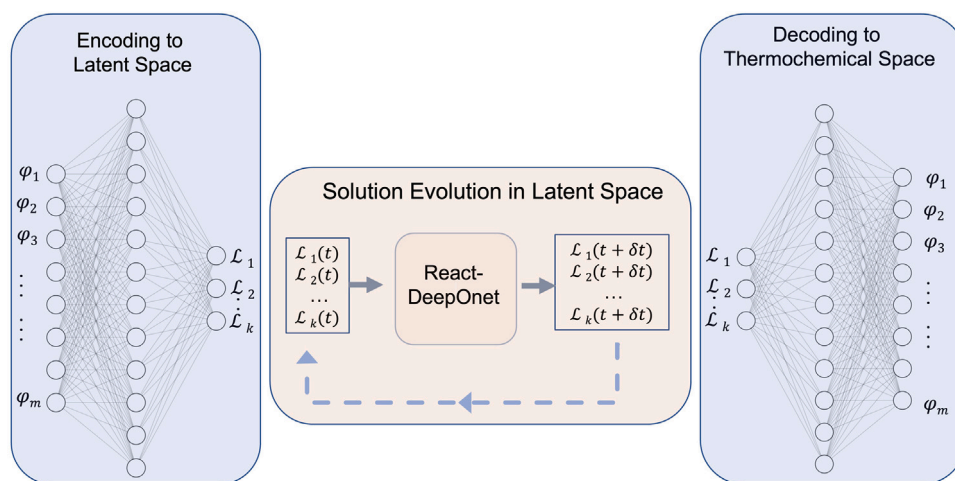


Fig. 5. A framework for kinetics identification and evolution in latent space: LS-React-DeepONet.

of  $k$  components. Training of the DeepONet using the latent space variables is similar to the training with representative scalars' vector of  $m$  components. The framework is shown in Fig. 5.

Note that the latent space variables are derived directly from the representative species and not from the full vector of thermochemical scalars, as illustrated in Fig. 4. One of the main applications of our modeling framework is to accelerate chemistry in CFD applications where chemistry is coupled with transport. Therefore, instead of transporting a large number of chemical species, we only account for the transport of the representative species. Thus, the LS-React-DeepONet is designed to map the solution of the representative scalars across time increments,  $\delta t$ .

Several advantages are associated with this framework of solution evolution in the latent space. First, learning the integration operator corresponding to only a few latent variables results in a smaller React-DeepONet architecture, as discussed in the Results section. This translates into faster training of the model and faster predictions from it. Second, the autoencoder, by projecting the thermochemical state to its latent state with the least number of variables, captures the essence of the identified kinetics and can potentially generalize well across different input conditions [57]. Finally, each condensed latent variable represents some specific characteristic or behavior of the reaction system enabling greater interpretability of the observed trends during fuel oxidation.

At this stage, we highlight the differences in the implementation of DeepONet in our work and the recent work of Goswami et al. [40]. Both approaches rely on a mapping procedure between two solution vectors separated by a time increment and use intermediate stages of the solutions as starting points for their training. Other key differences are summarized below:

- As opposed to our approach of training in windows of  $\Delta t$  by considering a wide range of time steps,  $\delta t$ , within the window, the work of Goswami et al. [40] relies on a small number, 4, of  $\delta t$ s within a given window.
- We rely in this study on the mapping of the solutions of a subset of the thermochemical scalars, the representative scalars. This is the same subset of species that will be transported within a CFD simulation. The work of Goswami et al. [40] maps the full solution vector (species and temperature). Scaling this procedure to complex fuels, without a latent space implementation, can be expensive in terms of constructing and training the DeepONet and its implementation within CFD. We believe that our reliance on representative scalars enables the implementation of our approach to complex fuels where the full chemical system is measured in terms of hundreds or thousands of species. In this

study, we validate our approach for a relatively simple fuel as well as n-dodecane to illustrate the performance of React-DeepONet for a range of fuel complexities.

- Both approaches use a latent space implementation for the input space via an encoder and decoder trained on the simulation data. In our approach, the latent space is constructed from the reduced set of representative scalars, while Goswami et al. [40] construct their latent space starting from the full chemical system.
- In our approach, we use a single DeepONet to prescribe the input vector of representative scalars. This approach is always manageable as we account only for a small fraction of the species in a full mechanism. In the work of Goswami et al. [40], two strategies were adopted: (1) different DeepONets are trained for individual species for canonical reaction mechanisms, and (2) a single DeepONet for their syngas-based simulations. An important advantage of using a single DeepONet is the ability to incorporate networks within DeepONet to enforce element and mass conservation, as proposed by Kumar and Echehki [39].
- In our implementation, the DeepONet is augmented with ParamNet, which helps encode parametric dependencies, such as equivalence ratio or pressure. It is also augmented by the ShiftNet, which is designed to improve the predictions of the DeepONet. Goswami et al. [40] propose the use of Partition-of-Unity (PoU), which enforces constraints on the outputs of the trunk net and is designed to enhance data interpolation.
- Finally, we also use long-time integration loss for refined training, which enhances the accuracy of predictions in a long time horizon.

#### 2.4. Data generation, processing and training in React-DeepONet

The proposed chemistry acceleration framework is data-based. Training of React-DeepONet is carried out offline using simulation data that spans the target composition space of the application on which the DeepONet is to be implemented. For demonstrating the framework in this paper, we rely on 0D homogeneous chemistry simulations with a range of prescribed initial mixture conditions, which result in temporal profiles of temperature and species concentrations. There are alternative strategies for generating the training data, which are briefly discussed in the Conclusions section. In the present study, solution profiles for each reaction system with different initial temperatures and equivalence ratios are obtained with Cantera [59] using a model for constant pressure, 0D, and homogeneous mixtures. For each fuel, the reaction evolves from an initial state until chemical equilibrium is reached after ignition. The solution vector comprising temperature and

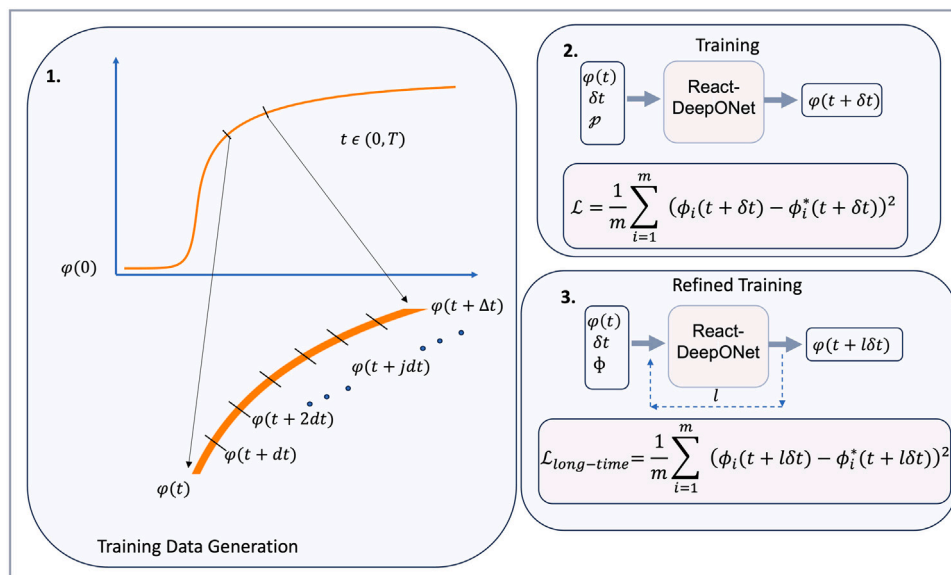


Fig. 6. React-DeepONet training framework.

species from the chemical mechanism is stored at a fixed time interval in each case.

To accommodate the disparate magnitudes of the various representative scalars and their variations in time and to incorporate the data into the network, data transformation is carried out:

- First, a fifth-order transformation is implemented on the species profiles to accommodate the strong multiscale variations of species mass fractions in the solution [60,61]. The transformation for a  $k$ th species mass fraction,  $Y_k$ , is implemented as follows:

$$y_k = Y_k^{1/5} \quad (11)$$

The same transformation is implemented for temperature. Similar transformations based on  $1/4$  and  $1/6$  powers yielded similar results.

- Second, the transformed variables,  $y_k$ , are normalized using their minimum and maximum values to yield values in the range of  $-1$  to  $1$ . This range is consistent with the implementation of the tangent sigmoid function as a transfer function in React-DeepONet. Other normalizations may be used to accommodate alternative transfer functions.

Once the variables to be evolved are identified and normalized to the desired range, input–output pairs are constructed as a training dataset. Part 1 of Fig. 6 shows a zoomed-in time window corresponding to a random solution vector,  $\phi(t)$ , on the profile. The input to the Branch Net corresponds to the vector of  $m$  representative scalars,  $\phi(t)$ , (or  $k$  latent space variables with LS-React-DeepONet). The input to the Trunk Net corresponds to factors of the time increments of the input,  $\delta t = j \, dt$ , up to a maximum step size of  $\Delta t$ . Here,  $j$  can vary from 0 to the integer value of  $\Delta t/dt$ . The output of the DeepONet corresponds to the solution vector of the representative scalars,  $\phi(t + j \, dt)$ , (or the latent space variables) at a time increment of  $j \, dt$ .

This window of  $\Delta t$  moves from the initial time,  $t = 0$ , to the final time. Therefore, any solution vector for the representative scalars at an intermediate time,  $t$ , can be used as training data and input to the Branch Net with different inputs to the Trunk Net that correspond to different time increments until  $t + \Delta t$  is reached.

The training data generation is performed on all the 0D trajectories and the parameter,  $p$ , associated with each 0D trajectory inputs to the ParamNet. This strategy allows the generation of a large training data

set with a limited number of 0D reactor simulations. Once the training data is generated, the loss function based on Mean Squared Error (MSE) is formulated, which can be expressed as follows:

$$\mathcal{L} = \frac{1}{m} \sum_{i=1}^m (\phi_i(t + \delta t) - \phi_i^*(t + \delta t))^2 \quad (12)$$

where  $\phi_i^*(t + \delta t)$  is the ground truth solution vector. After initial training using the aforementioned loss function as shown in part 2 of Fig. 6, we proceed to a refined training phase of the model which is shown in part 3 of Fig. 6. The refined training employs a novel loss function formulation, which leverages iterative predictions made by the model. For a particular  $\phi(t)$ , the model advances the solution vector by  $\delta t$  and outputs  $\phi(t + \delta t)$ . The output from the model,  $\phi(t + \delta t)$ , can in turn serve as input to the Branch Net, and the resulting model output,  $\phi(t + 2\delta t)$ , is advanced by  $2\delta t$ . This can be done  $l$  times iteratively resulting in  $\phi(t + l\delta t)$ , which corresponds to advancing the solution vector by  $l\delta t$ . Here, instead of minimizing the loss associated with just the first prediction  $\phi(t + \delta t)$ , this loss, instead, is minimized for the  $l$ th prediction. The expression for the corresponding loss function, then, maybe written as follows:

$$\mathcal{L}_{long-time} = \frac{1}{m} \sum_{i=1}^m (\phi_i(t + l\delta t) - \phi_i^*(t + l\delta t))^2 \quad (13)$$

In our study, we find that the implementation of this iterative solution procedure for minimizing the loss function works well for values of  $l$  up to 5. It tends to reduce the overall prediction error over long integration times.

## 2.5. Reconstruction of the remaining thermochemical scalars

Both React-DeepONet and LS-React-DeepONet rely on the mapping of a subset of the species of the chemical system or variables in its latent space. The premise of the two approaches is that other species of interest, which are not solved in DeepONet, can be reconstructed from the modeled scalars. In this study, we use ANN regression to relate the resolved scalars with the remaining ones. The training for these ANNs is implemented using the training data on which DeepONets are also constructed.

Figs. 7(a) and 7(b) illustrate the architecture of the ANNs used to recover the remaining scalars from knowledge of the representative

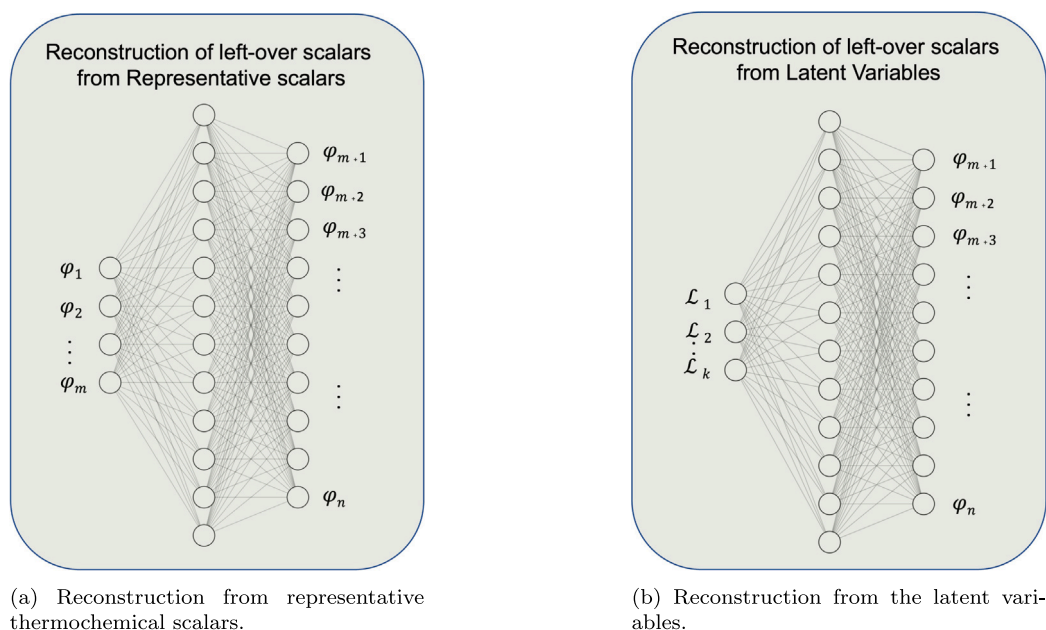


Fig. 7. Reconstruction of remaining thermochemical scalars.

scalars or the latent space variables, respectively. In these networks, the  $m$  representative scalars or the  $k$  latent variables are prescribed as inputs to these ANNs, respectively. The output corresponds to the vector of the remaining scalars we wish to predict. As reported below, the required ANNs for reconstructing the remaining scalars are relatively shallow as they feature only 2 hidden layers. This relatively shallow architecture reflects the nature of the representative scalars or their low-dimensional representation through the latent variables as adequate parameters to characterize the entire chemical system.

### 3. Results

The performance of the two DeepONet frameworks, React-DeepONet and LS-React-DeepONet, is demonstrated in this section. React-DeepONet is used to predict the oxidation of hydrogen and the low-temperature (LTO) oxidation of a more complex fuel, n-dodecane. For brevity, the LS-React-DeepONet is demonstrated for n-dodecane LTO.

LTO for complex fuels, which typically occurs at temperatures below 1000 K, exhibits significant complexities over the corresponding high-temperature oxidation (HTO) for the same fuels. During HTO (typically above 1000 K), the early stages of oxidation are governed by fast pyrolysis, resulting in the formation of a handful of fragments, which in turn are oxidized at a much slower pace [62]. Typical pyrolysis fragments tend to be common among a wide range of complex hydrocarbon fuels [62]. In contrast, the LTO of the same fuels is characterized by the initial formation of many fuel-specific intermediates [63] and the presence of 2 stages of the ignition process. Capturing these chemical dynamics is a challenge for the development of reduced and skeletal mechanisms for LTO.

As outlined earlier, we adopt a 0D homogeneous chemistry model at constant pressure to study the evolution of representative species in time and recover the remaining species if needed. The training can be extended to conditions where transport is present. Within the context of species' evolution involving transport, one common strategy is to adopt a split operator approach where chemistry and transport are integrated separately and corrections for the operator-splitting errors are implemented [64].

Table 1 lists the different React-DeepONet and the LS-React-DeepONet models and training hyper-parameters, which were implemented to predict the oxidation of  $H_2$  and n-dodecane. The table

shows the neural network size for the Branch Net, the Trunk Net, the Shift Net, and the ParamNet for different React-DeepONet models. As expected, the React-DeepONet models for the complex fuel require a large number of training parameters for n-dodecane LTO. Also, note that a significant reduction in the network models is obtained with LS-React-DeepONet.

As illustrated in the table, the ShiftNet architecture is significantly smaller relative to the overall DeepONet architecture and the associated training parameters are around 80 to 100 times less than Branch Net and Trunk Net. Yet, as shown in Section 3.1, it does contribute to significantly lower training loss. The ParamNet, 3 to 4 times smaller than Branch Net and Trunk Net, is used in this study to accommodate variations in the equivalence ratio in the training data. As a result, as shown in Section 3.2, predictions from React-DeepONet become significantly more accurate. Moreover, it can also be used to accommodate variations in other key parameters, like pressure or dilution.

The React-DeepONet architecture is modeled and trained with the JAX [65] framework utilizing Just-in-time compilation and auto-vectorization on an NVIDIA RTX A5500 GPU to reduce the training time significantly. The model is trained with the Adamax optimizer [66]. The learning rate, listed in Table 1, exponentially decays with a step of 5000 and a rate of 0.9 in all cases.

#### 3.1. $H_2$ oxidation

React-DeepONet is first evaluated for  $H_2$  oxidation. For the training purpose, solution profiles are generated from Cantera [59] with the 0D constant pressure homogeneous reactor. The hydrogen mechanism by Burke et al. [67] is used for modeling  $H_2$  oxidation and for generating the training data. The mechanism includes 10 species and 20 reactions. With the set of reactions determined by the reaction mechanism, the solution vector, consisting of species mass fractions and temperature, is evolved in time for initial temperatures  $T_i = 980, 1000, 1020, 1040, 1060, 1080, 1100, 1120, 1140, 1160, 1180, 1200, 1220, \text{ and } 1240$  K and equivalence ratios  $\Phi = 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95, 1.0, 1.05, 1.1, 1.15, 1.2, 1.25, \text{ and } 1.3$  such that a total of 210 solution profiles are generated, with only a subset is used for training the React-DeepONet. The React-DeepONet model is trained on  $\Phi = 0.75, 0.8, 0.85, 0.9, 0.95, 1.0, 1.05, 1.1, \text{ and } 1.15$  equivalence ratios and  $T_i = 1000, 1020, 1040, 1060, 1080, 1100, 1120, 1140, 1160, 1180$  and

**Table 1**  
React-DeepONet models and training parameters.

| Parameters                        | H <sub>2</sub>   | React-DeepONet<br>n-Dodecane | LS-<br>React-DeepONet |
|-----------------------------------|------------------|------------------------------|-----------------------|
| Training window ( $\Delta t$ )    | (40 $\mu$ s)     | (100 $\mu$ s)                | (100 $\mu$ s)         |
| Branch Net                        | [5,50,50,50,200] | [8,100, ..., 100,480]        | [3,50, ..., 50,180]   |
| Trunk Net                         | [1,50,50,50,200] | [1,100, ..., 100,480]        | [1,50, ..., 50,180]   |
| ShiftNet                          | [5,10,10,1]      | [8,40,40,1]                  | [3,10,10,1]           |
| ParamNet                          | [1,50,200]       | [1,100,480]                  | [1,50,50,180]         |
| Total parameters                  | 42 k             | 250 k                        | 94 k                  |
| Training iterations               | $1 \times 10^5$  | $9 \times 10^4$              | $9 \times 10^4$       |
| Refined training iterations       | $2 \times 10^4$  | $2 \times 10^4$              | $2 \times 10^4$       |
| Total training time (GPU minutes) | 12.5             | 30.0                         | 25.5                  |
| Mini-Batch size                   | 20,000           | 30,000                       | 30,000                |
| Learning rate                     | $10^{-2}$        | $8 \times 10^{-3}$           | $8 \times 10^{-3}$    |

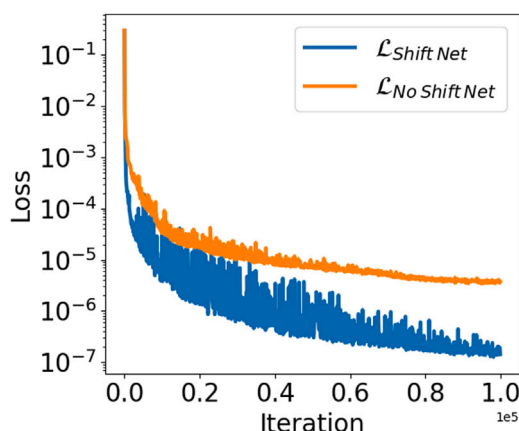


Fig. 8. Loss with and without ShiftNet.

1200 K initial temperatures. The model is tested on the rest of the equivalence ratios and initial temperatures for extrapolation.

The pressure is fixed at 1 atm for all cases and the solution is advanced to a final time of 1.0 ms. The solution vector is stored at fixed time steps of  $10^{-7}$  s such that a total number of  $10^4$  solution vectors are generated for each set of initial conditions. Five representative thermochemical scalars, including  $T$  and the mass fractions of H<sub>2</sub>, H, O<sub>2</sub>, and H<sub>2</sub>O, are selected out of the entire solution vector containing 11 thermochemical scalars using the procedure outlined above. This full set includes temperature and the mass fractions of H<sub>2</sub>, H, O<sub>2</sub>, O, OH, H<sub>2</sub>O, HO<sub>2</sub>, H<sub>2</sub>O<sub>2</sub>, AR, and N<sub>2</sub>. A time window of  $\Delta t = 40 \mu$ s is chosen to learn the solution operator with React-DeepONet such that the maximum time advancement in a single prediction is 40  $\mu$ s. The results are presented for a time step,  $\delta t$ , of 20  $\mu$ s.

Fig. 8 illustrates the potential benefit of implementing a ShiftNet within React-DeepONet during training for H<sub>2</sub> oxidation by comparing the MSE losses with and without the ShiftNet. The training losses are significantly reduced with the ShiftNet.

Fig. 9 shows the temporal evolution of temperature and the representative species mass fraction at different initial temperatures and equivalence ratios. Note that all temporal profiles in the figures below are based on the  $1/5$  power transformation given in Eq. (11). Then, the resulting transformation is normalized using minimum and maximum values of these transformed variables to yield a range from 0 to 1. The choice of the transformation to present results is due to the ability of the transformation to better illustrate comparisons for values near the minimum and maximum values of the solutions [60,61]. The normalization is also used to plot profiles on the same plots. The plots will refer to the transformation results and be normalized by simply using “normalized”. Regardless, these plots are used for comparisons between

DeepONet predictions and those resulting from the direct integration of the solutions using Cantera.

All the plots compare normalized values of temperature and mass fractions of H<sub>2</sub> and H. It can be observed that the predictions from React-DeepONet achieve an excellent agreement with the solution profiles from Cantera for a range of initial conditions with the MAEs being less than 0.1%. React-DeepONet accurately captures the chemical kinetics of the H<sub>2</sub> oxidation where ignition time and chemical equilibrium are well-matched with the corresponding actual values.

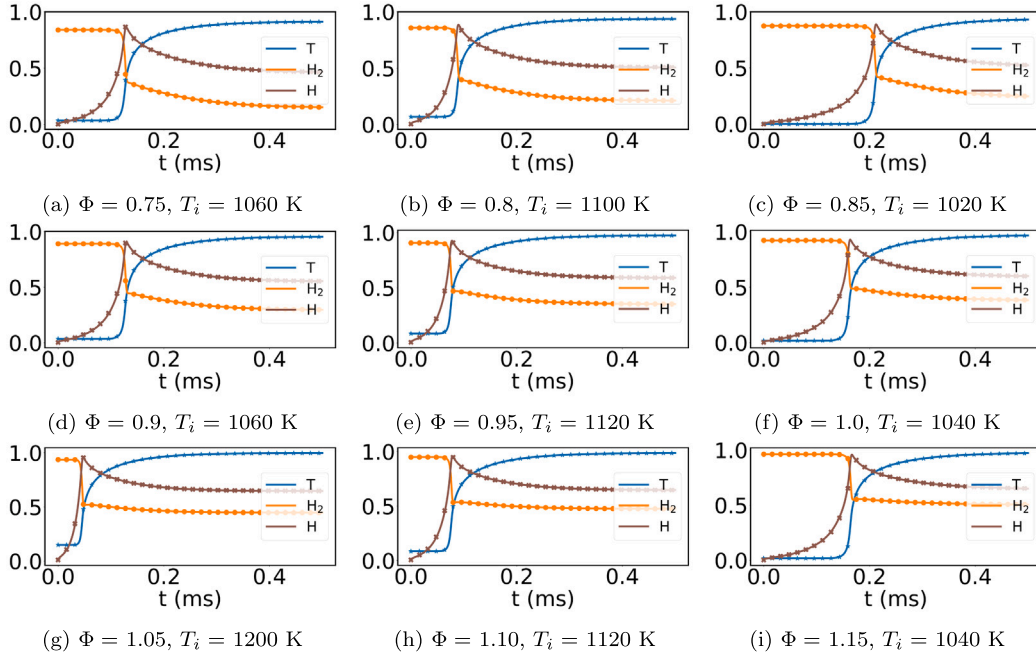
Figs. 10 and 11 illustrate the extrapolation capability of React-DeepONet. The figures show *a posteriori* predictions of the temporal evolution of temperature and species mass fractions with equivalence ratios and initial temperature, respectively, outside the training conditions. The extrapolation for the equivalence ratio is within  $\pm 0.15$ , while the extrapolation range for initial temperature is within  $\pm 40$  degrees. It is important to note that the comparisons of results involving conditions outside the original training data are presented for H<sub>2</sub> and n-dodecane oxidation to show the robustness of the DeepONet architecture. When training React-DeepONet, it is important to select training data to reproduce the desired composition space in *a posteriori* its implementations.

As shown in Fig. 10, React-DeepONet extrapolates well across equivalence ratios, and predictions are in very good agreement with the predicted solution from Cantera with mean absolute errors (MAE) less than 0.5% for all the cases. The model also extrapolates reasonably well for different initial temperatures as shown in Fig. 11 with a maximum MAE of 4.0%. The model predictions are close to the predicted solutions from Cantera except for the lowest initial temperature of  $T_i = 980$  K where an early ignition is observed.

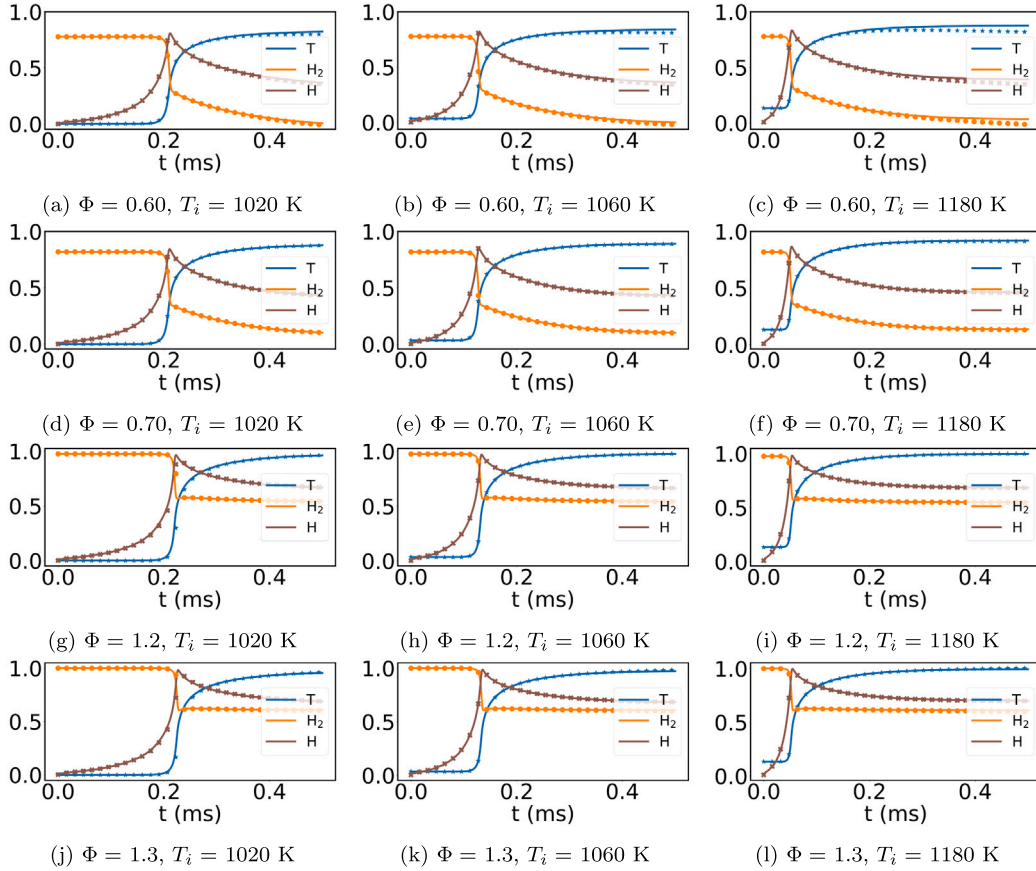
To illustrate the performance of React-DeepONet across all the initial conditions, Fig. 12 shows the variation of the prediction error in the form of normalized MAE contours across different initial conditions. The central box corresponds to the training zone and has initial conditions of interpolation and the rest of the initial conditions outside the box are tested for extrapolation.

Across the entire parametric space of initial conditions, the errors remain within less than 4.0% in the extrapolation zone and less than 0.1% in the interpolation zone. As expected, the errors are smallest in the middle of the training zone and are slightly larger at its boundaries. A similar trend is observed in the extrapolation zone as well where errors increase when initial conditions move away from the training zone. The reasonable performance of React-DeepONet, especially for initial temperature extrapolation, is remarkable given the strong non-linear dependence of reaction on temperature during all stages of the oxidation process.

Fig. 13 shows the temporal evolution of denormalized temperature and H<sub>2</sub> mass fraction on the actual scale for different initial temperatures and at  $\Phi = 0.95$ . Again, similar to the above plots, an excellent agreement between the predictions from React-DeepONet and Cantera is observed.



**Fig. 9.** (Hydrogen oxidation) Temporal evolution of the normalized temperature and mass fractions. The solid line and symbols represent Cantera and React-DeepONet solutions respectively.



**Fig. 10.** (Hydrogen oxidation) Extrapolation capabilities of React-DeepONet with respect to the equivalence ratio. Solid line: Cantera solution; symbols: React-DeepONet.

Fig. 14 demonstrates the flexibility in selecting the time step,  $\delta t$ , for solution advancement in React-DeepONet. Any time step in  $\delta t$  within the training window size of  $\Delta t$  can be adopted. Furthermore, the solution advancement does not have to be at a fixed time step. At

any point on the solution evolution trajectory, the time step can be adaptively adjusted to capture slow or fast variations of the solution profiles. Figs. 14(a) and 14(b) compare the solution vector advanced at constant time-steps of  $8.0 \mu s$  and  $40.0 \mu s$ , respectively. On the other

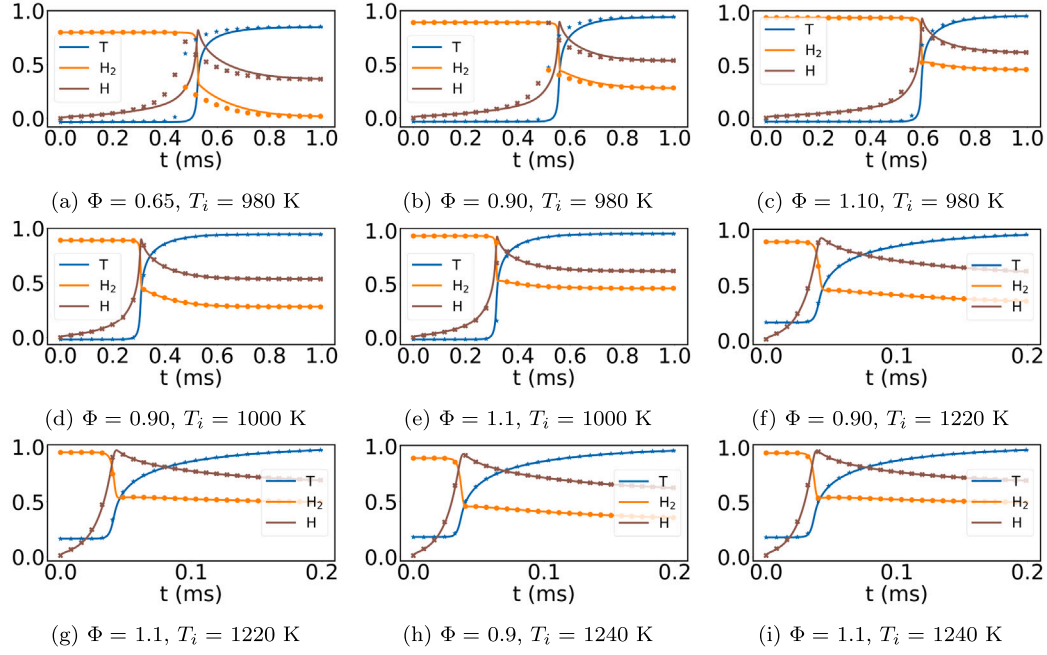


Fig. 11. (Hydrogen oxidation) Extrapolation capabilities of React-DeepONet with respect to the initial temperature. Solid line: Cantera solution; symbols: React-DeepONet.

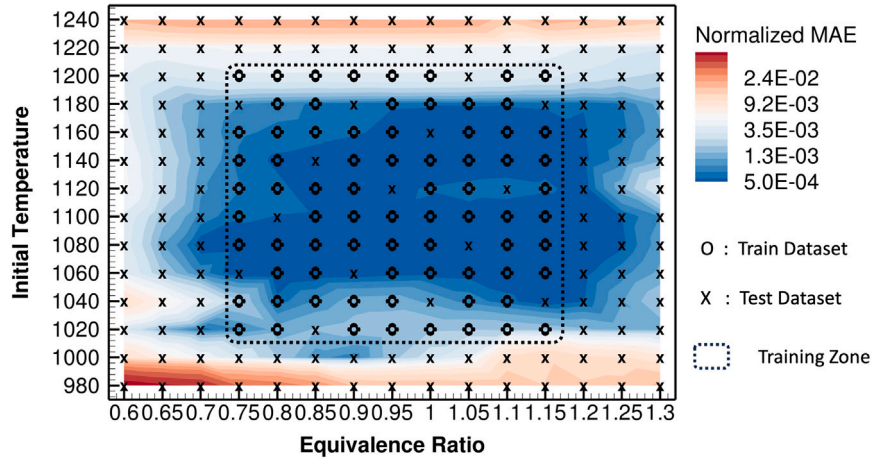


Fig. 12. (Hydrogen oxidation) Variation of React-DeepONet prediction error with different initial conditions.

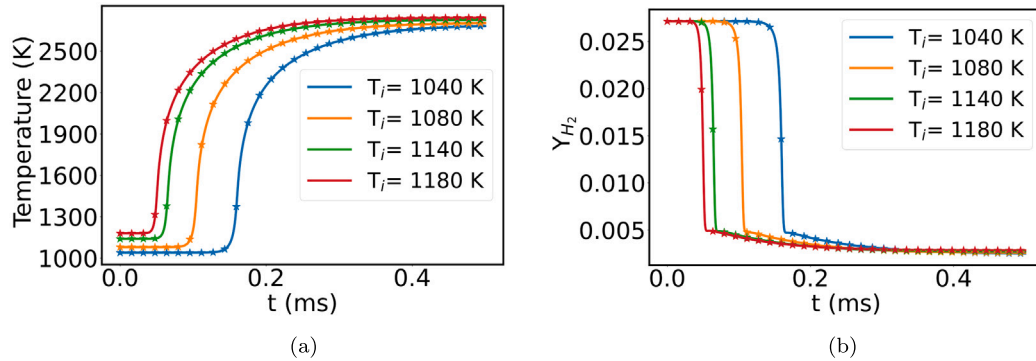


Fig. 13. (Hydrogen oxidation) Temporal evolution of denormalized temperature and  $H_2$  mass fraction at  $\Phi = 0.95$  and various initial temperatures. The solid line and symbols represent Cantera and React-DeepONet solutions, respectively.

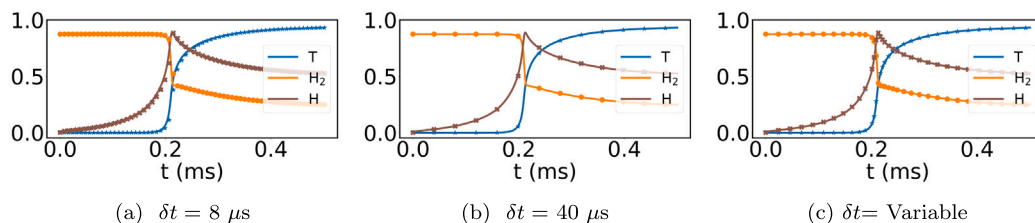


Fig. 14. (Hydrogen oxidation) Predictions of temperature and  $H_2$  and  $H$  mass fractions using React-DeepONet at different time steps,  $\delta t$  ( $\Phi = 0.85$ ,  $T_i = 1020$  K).

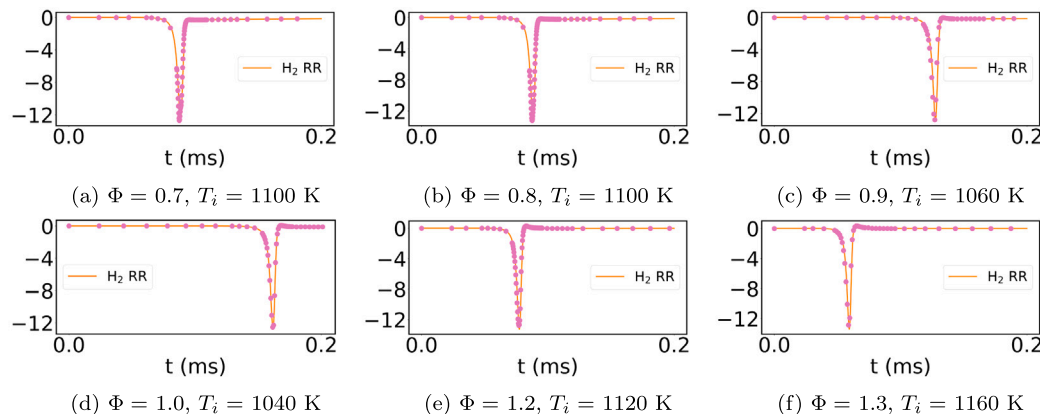


Fig. 15. (Hydrogen oxidation) Source terms for normalized  $H_2$ . Solid line: Cantera solution; symbols: React-DeepONet.

hand, in Fig. 14(c), the solution vector is advanced at a variable time step based on the temperature source terms as shown below:

$$dt = \frac{\Delta t}{2(1 + |S_{T_n}|)} \quad (14)$$

where  $S_{T_n}$  is the source term for normalized temperature.

Leveraging the automatic gradient functionality of JAX [65], the network output can be partially differentiated with time, based on Eq. (4). Fig. 15 compares these source terms for normalized  $H_2$  mass-fraction from React-DeepONet and from Cantera's detailed simulations, demonstrating a very good agreement. Therefore, React-DeepONet can also predict source terms in addition to its ability to predict solutions along temporal profiles. Nonetheless, it should be noted that the present React-DeepONet's primary purpose is to map the solution of the thermochemical state in small time increments. It is possible to consider scenarios where reaction source terms may be needed during a simulation. In those scenarios, source term losses may be included during the minimization of the losses during React-DeepONet training to improve the predictions of the chemical source terms.

### 3.2. n-Dodecane (Low-temperature oxidation)

In this section, React-DeepONet and LS-React-DeepONet are demonstrated for the more complex chemical reaction system for n-dodecane LTO. The JetSurF2.0 [68] reaction mechanism consisting of 348 species and 2163 reactions is used to generate training data and for the validation of the two DeepONet methods. During n-dodecane LTO, fuel-specific species are produced during the early oxidation steps, subsequently yielding the first and second stages of ignition [63].

To study the LTO of n-dodecane, the homogeneous ignitions of 14 initial temperatures from  $T_i = 730$  to  $860$  K at  $10$  degrees increments and  $15$  equivalence ratios,  $\Phi = 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95, 1.0, 1.05, 1.1, 1.15, 1.2, 1.25$ , and  $1.3$ , are simulated. The React-DeepONet model for n-dodecane LTO is trained on  $\Phi = 0.75, 0.8, 0.85, 0.9, 0.95, 1.0, 1.05, 1.1$ , and  $1.15$  equivalence ratios and  $T_i = 750, 760, 770, 780, 790, 800, 810, 820, 830$  and  $840$  K initial temperatures. The model is tested on the rest of the equivalence ratios and initial

temperatures for extrapolation. All simulations in Cantera are carried out at a constant pressure of  $20$  atm and are advanced to a final time of  $10$  ms. The solution vector is stored at fixed time increments of  $5 \times 10^{-7}$  s such that a total of  $20,000$  solution vectors are generated for each case. The results are presented for a time step,  $\delta t$ , of  $100 \mu s$ , which also corresponds to the full window of training of  $\Delta t = 100 \mu s$ .

Given the inherent complexity of n-dodecane LTO, a larger set of representative scalars is used compared to  $H_2$  oxidation. These scalars, which are identified using the procedure discussed above [20], include temperature and the mass fractions of  $CO_2$ ,  $H_2O$ ,  $H_2$ ,  $CO$ ,  $HCCO$ ,  $C_2H_4$  and  $C_4H_8 - 1$ . This subset is selected out of a total of  $349$  thermochemical scalars. The time window of  $\Delta t = 100 \mu s$  with  $200$  solution points is chosen for training the DeepONets. Table 1 provides a detailed overview of the network architecture and training hyper-parameters employed in the React-DeepONet model, which is implemented to learn the chemical kinetics of n-dodecane LTO.

Fig. 16 shows the temporal evolution of the normalized values of temperature and the mass fractions of  $CO$  and  $C_4H_8-1$  for the different initial temperatures and equivalence ratios. The figure is based on the use of ParamNet in optimizing React-DeepONet. The figures yield good predictions of the profiles with a maximum MAE of  $3.1\%$ .

Fig. 17 illustrates the React-DeepONet predictions without ParamNet for the equivalence ratio keeping the total number of trainable weights the same as the results shown with ParamNet in Fig. 16. The figure clearly shows that the use of ParamNet can significantly enhance the predictions of React-DeepONet. Although the n-Dodecane LTO trend is somewhat captured without ParamNet, ignition delays and equilibrium values are not matched. These results reinforce the contribution of ParamNet in the React-DeepONet model employed for complex fuel.

Figs. 18 and 19 illustrate the extrapolation capability of React-DeepONet for n-dodecane LTO for the equivalence ratio and the initial temperature, respectively. The plots show the temporal evolution of temperature and species mass fractions at equivalence ratios that are entirely outside of the training zone. The extrapolation for the equivalence ratio is within  $\pm 0.15$ , while the extrapolation range for initial temperature is within  $\pm 20$  degrees.

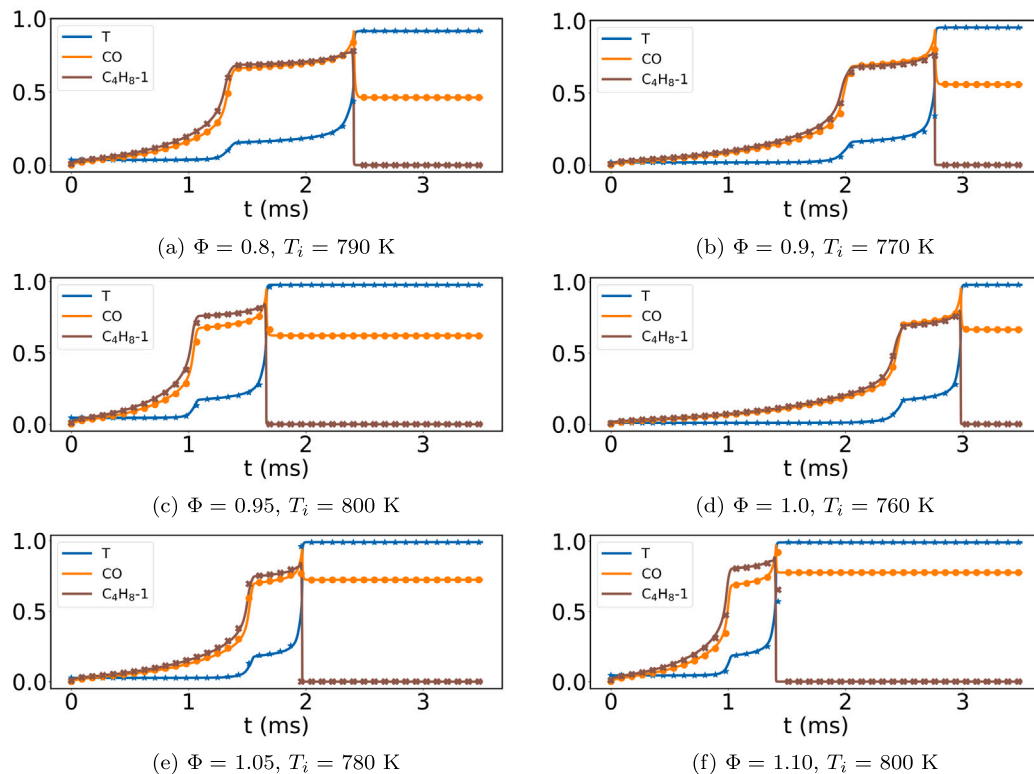


Fig. 16. (n-Dodecane LTO) Temporal evolution of the normalized temperature and mass fractions with ParamNet. Solid line: Cantera solution; symbols: React-DeepONet.

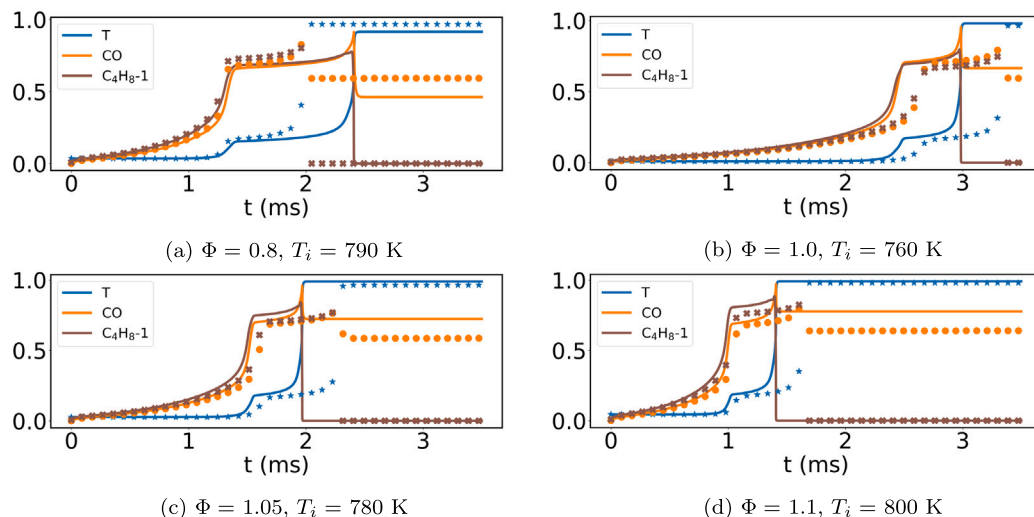


Fig. 17. (n-Dodecane LTO) Temporal evolution of the normalized temperature and mass fractions without ParamNet. Initial conditions similar to those in Fig. 16-(a,d,e,f), Solid: Cantera solutions, Symbols: React-DeepONet.

Fig. 18 shows that, across different equivalence ratios, the React-DeepONet predictions are in very good agreement with the predicted solutions from Cantera with MAEs being less than 5.0% for all the cases. On the other hand, the extrapolation for the initial temperature exhibits a less satisfactory performance for the low initial temperature of 730 K, as shown in Fig. 19, with a maximum MAE of around 9.5%. Training with an increased number of finer temperature increments at the low-temperature boundary of the training range can improve the predictions of the lower temperature extrapolation. Yet, it remains important to select training data that targets the desired conditions for which React-DeepONet will be applied.

Similar to the parametric error plot for  $\text{H}_2$  oxidation in Fig. 12, Fig. 20 shows the variation of the prediction error in the form of normalized MAE contours across different initial conditions for n-Dodecane LTO. Across the entire parametric space of initial conditions, the errors remain within less than 3.1% in the interpolation zone and 10.0% in the extrapolation zone. Like the  $\text{H}_2$  oxidation case, the model performs better in the high initial temperature and fuel-rich extrapolation zone than in the low initial temperature and fuel-lean extrapolation zone. In addition to that, extrapolation in equivalence ratio is better as compared to the extrapolation in the initial temperature which degrades faster when initial temperatures become smaller. Even for the least accurate predictions with MAEs of approximately 8.0%, the

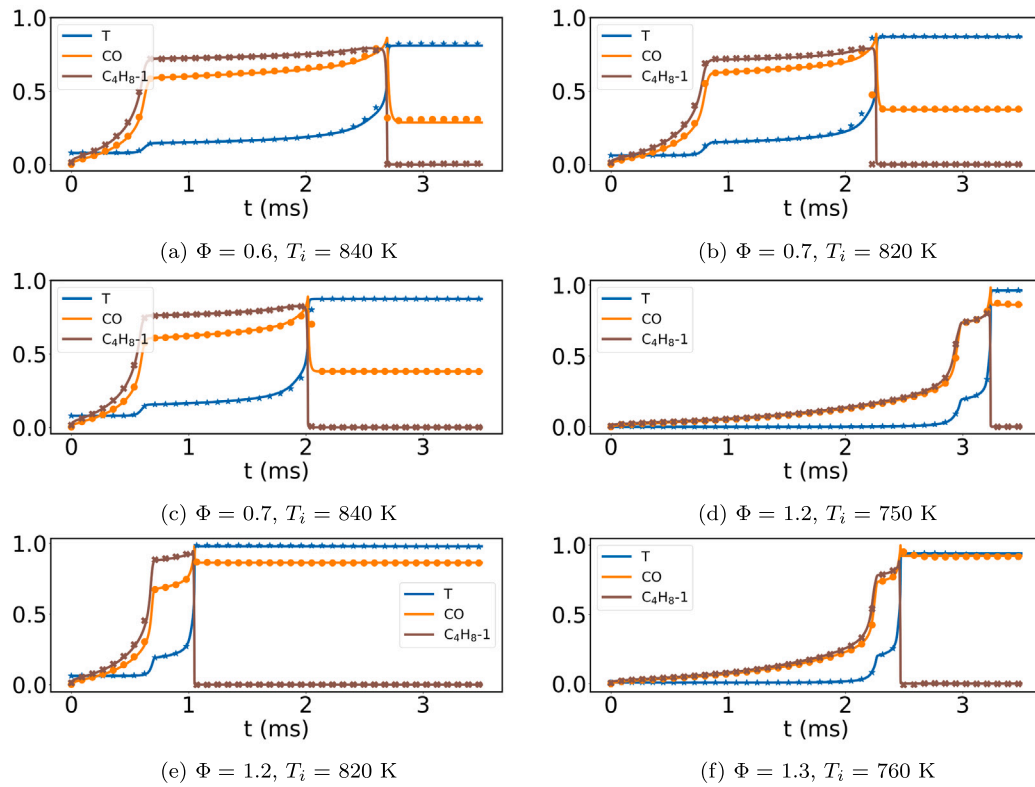


Fig. 18. (n-Dodecane LTO) Extrapolation capabilities of React-DeepONet with respect to equivalence ratio. Solid line: Cantera solution; symbols: React-DeepONet.

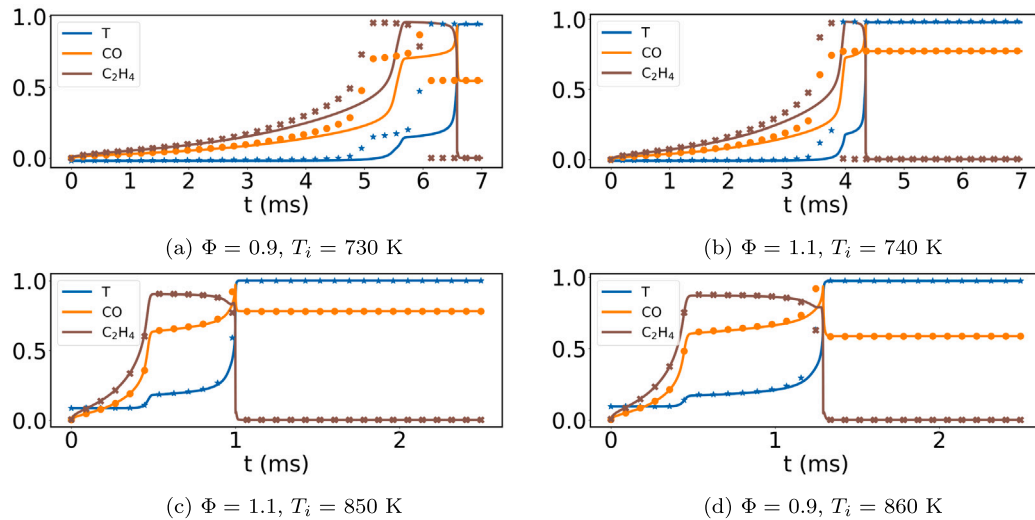


Fig. 19. (n-Dodecane LTO) Extrapolation capabilities of React-DeepONet with respect to the initial temperature. Solid line: Cantera solution; symbols: React-DeepONet.

React-DeepONet predictions in Figs. 19(a)–19(b) qualitatively track the trends of n-Dodecane LTO with a slight shift in the ignition delays.

Fig. 21 shows the temporal evolution of the temperature and  $C_4H_8-1$  mass fraction without the normalization presented so far for various initial temperatures at  $\Phi = 0.95$ . As argued earlier, the  $1/5$  power transformation tends to accentuate the variation in the solution profiles near the minimum and maximum values of the transformed variables. Again, similar to  $H_2$  oxidation, n-Dodecane LTO denormalized values on the actual linear scale are also in good agreement.

The remaining thermochemical scalars, which are not part of the

representative scalars, are highly correlated with the representative scalars and can be non-linearly reconstructed through an ANN as a function of representative scalars. A standard neural network with two hidden layers each having 40 nodes is used for this task.

Fig. 22 shows the temporal evolution of the normalized reconstructed species profiles at an initial temperature and equivalence ratio of 790 K and 0.8, respectively. Even though these species are not predicted by React-DeepONet, their strong correlations with the represented species enabled their adequate predictions through a separate ANN.

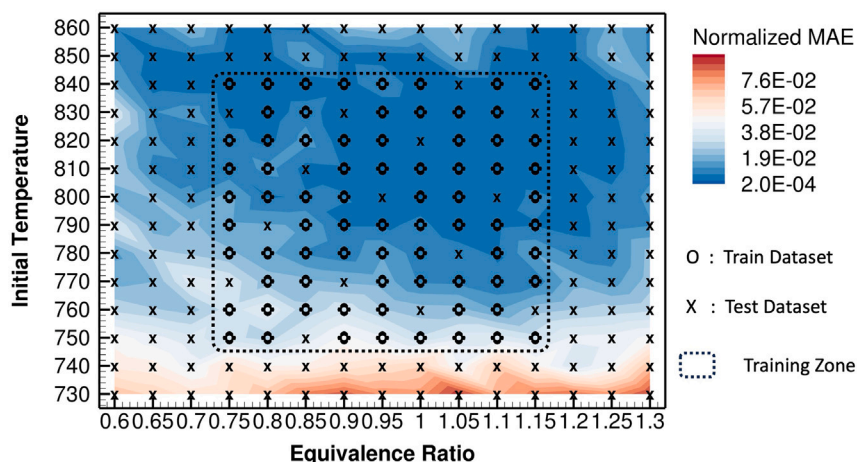


Fig. 20. (n-Dodecane LTO) Demonstrating variation of React-DeepONet prediction error with different initial conditions.

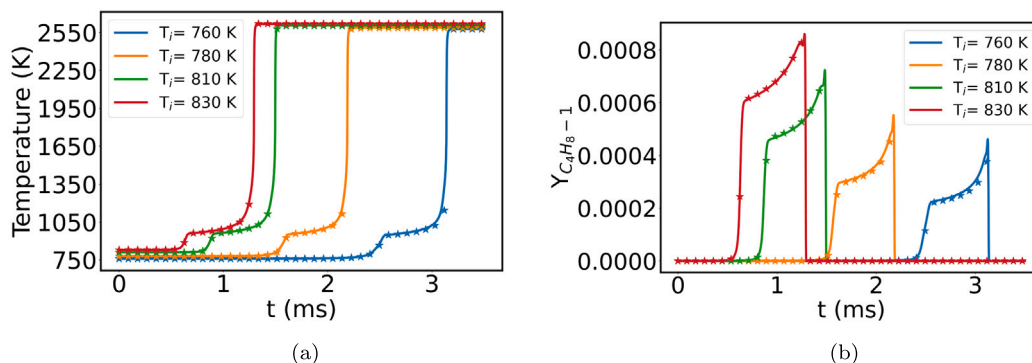


Fig. 21. (n-Dodecane LTO) Temporal evolution of denormalized temperature and  $C_4H_8-1$  mass fraction at  $\Phi = 0.95$  and various initial temperatures. The solid line and symbols represent Cantera and React-DeepONet solutions, respectively.

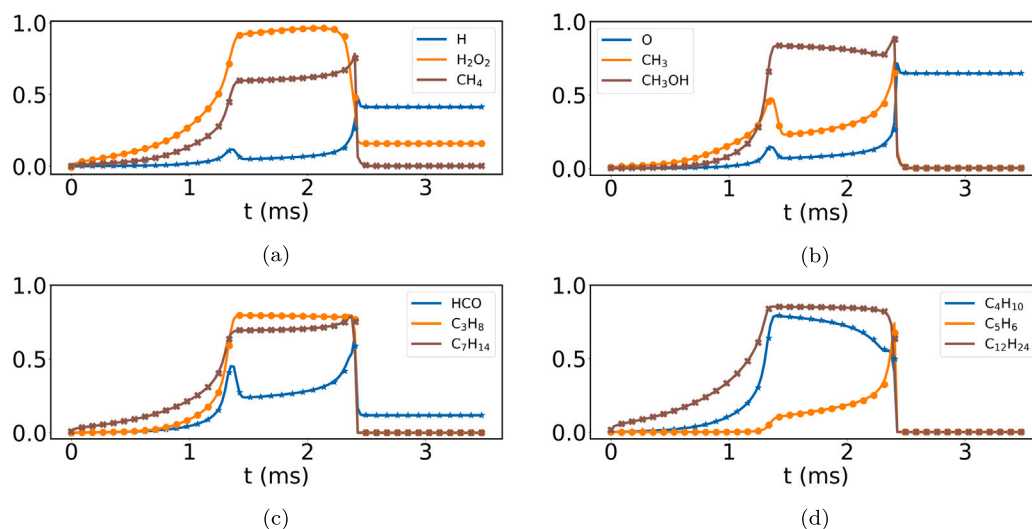


Fig. 22. (n-Dodecane LTO) Reconstructed remaining scalars from representative scalars at  $\Phi = 0.8$ ,  $T_i = 790.0$  K. Solid: Cantera solutions, Symbols: React-DeepONet.

### 3.2.1. Latent space kinetics identification

As stated earlier, the React-DeepONet size increases with the number of representative scalars and the complexity of the oxidation chemistry. A larger set of representative scalars is needed for a more complex combustion reaction system. One approach to mitigate the potential

cost of increasing the number of representative scalars is to use LS-React-DeepONet. The vector of latent variables, obtained from the autoencoder, is inherently smaller than that of the representative scalars or the total number of species in a chemical mechanism [57].

To validate the LS-React-DeepONet approach, the same training dataset for n-dodecane LTO is used. The 8 scalars are condensed

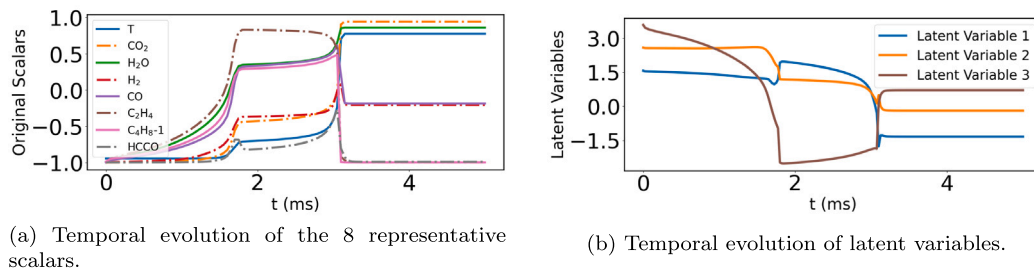


Fig. 23. (n-Dodecane LTO with LS-React-DeepONet) Dimensionality reduction of the representative set to latent variables.

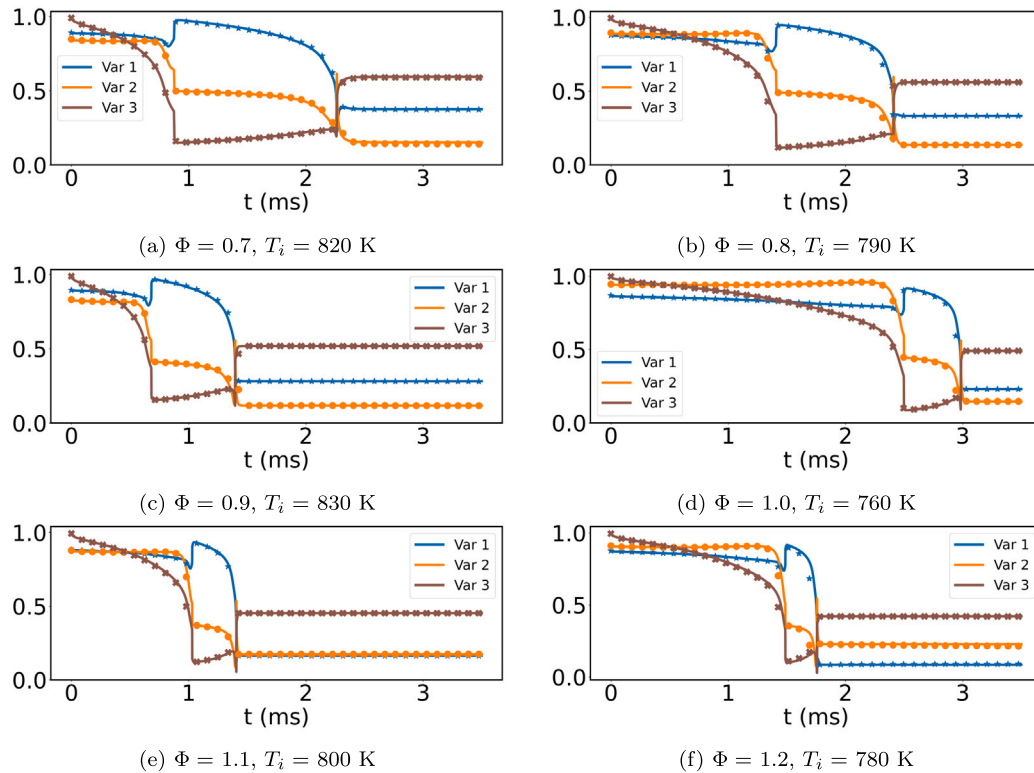


Fig. 24. (n-Dodecane LTO with LS-React-DeepONet) Temporal evolution of the normalized latent variables. Solid: Cantera solutions, Symbols: LS-React-DeepONet.

down to 3 latent variables (i.e., 3 nodes in the bottle-neck) with an autoencoder of size [8, 60, 20, 3, 20, 60, 8]. The network reconstructs those 8 scalars as an output. The latent variables can represent all 8 thermochemical scalars with a mean squared error of  $6.3 \times 10^{-6}$ . Fig. 23 shows the temporal variation of all the representative thermochemical scalars and their corresponding latent variables for a single solution profile. The figure shows that the 3 retained latent space variables capture the two-stage ignition process characteristic of LTO chemistry. Also, these variables exhibit profiles that represent the evolution of the representative scalars from which these variables are constructed.

Here, LS-React-DeepONet is trained on the latent variables instead of representative thermochemical scalars. The evolution of a smaller solution vector requires a smaller operator size. The size of the LS-React-DeepONet to evolve the latent variables is less than half of the one needed for React-DeepONet. The model parameters and training hyper-parameters are listed in Table 1.

Fig. 24 shows the temporal evolution of the normalized latent variables at different initial temperatures and equivalence ratios. The inferred solution from LS-React-DeepONet shows an excellent agreement with the reduced latent variables from Cantera. The latent variables also exhibit the two-stage ignition patterns characteristic of n-dodecane LTO. They also represent the contributions and variations in the profiles of the underlying representative scalars.

Fig. 25 shows the temporal profiles of the normalized thermochemical scalars, which are reconstructed from the latent variables. Fig. 25(a) shows the reconstruction of mass fractions which are part of the representative set. This reconstruction is performed through the decoder layer of the autoencoder. In contrast, Figs. 25(b)–25(d) shows the reconstruction of non-representative scalars, which are not part of the representative thermochemical scalars. This reconstruction is performed through an ANN (with two hidden layers each having 40 nodes), which provides a mapping from latent variables to the sample of remaining thermochemical scalars as shown in Fig. 25. Both the representative and remaining thermochemical scalars are in good agreement with the Cantera predictions. The reconstruction of desired thermochemical scalars from the latent variables at any point during the solution evolution makes this approach comprehensive.

### 3.3. Speed-up

Table 2 summarizes the training time and speed-up associated with each case of React-DeepONet. The table also lists the time taken to infer a solution on the entire profile from both React-DeepONet and Cantera. The solution vector is integrated to 0.5 ms for  $\text{H}_2$  combustion and 5 ms for n-dodecane LTO. To get the solution profile from React-DeepONet,

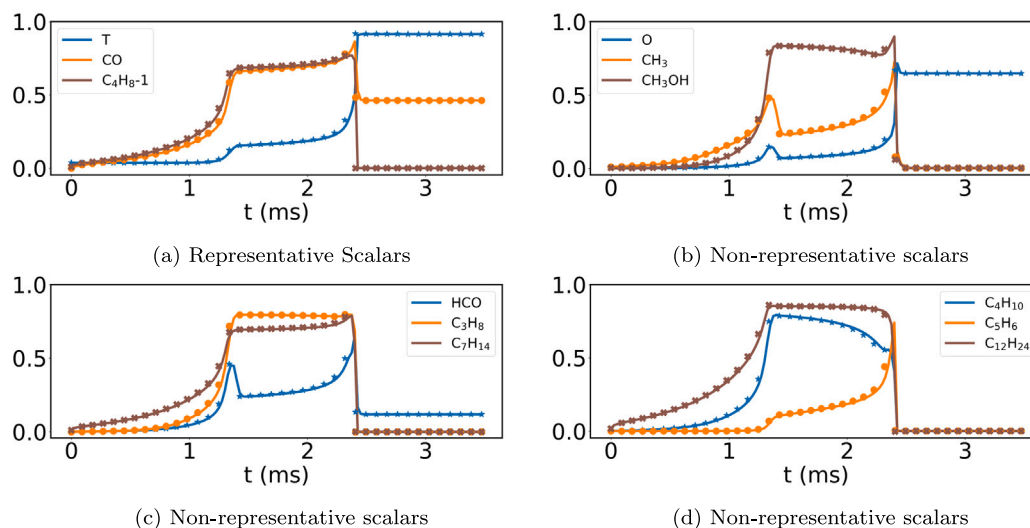


Fig. 25. (n-Dodecane LTO with LS-React-DeepONet) Reconstructed representative and remaining scalars from latent variables at  $\Phi = 0.8$ ,  $T_i = 790.0$  K. Solid: Cantera solutions, Symbols: LS-React-DeepONet.

Table 2  
Speed-up with React-DeepONet integration.

| Parameters                                     | H <sub>2</sub> | Low-Temp<br>n-Dodecane | LS-<br>React-DeepONet |
|------------------------------------------------|----------------|------------------------|-----------------------|
| Final time on solution trajectory ( $t$ ) (ms) | 0.5            | 5                      | 5                     |
| Integration time step ( $\delta t$ )           | 20 $\mu$ s     | 100 $\mu$ s            | 100 $\mu$ s           |
| Total parameters                               | 42 k           | 250 k                  | 94 k                  |
| React-DeepONet integration CPU time (ms)       | 0.52           | 3.30                   | 2.00                  |
| Cantera integration CPU time (ms)              | 6.5            | 5069                   | 5069                  |
| Speed-Up                                       | 12.5           | $\approx 1500$         | $\approx 2500$        |

the maximum integration time step is considered in each case without losing prediction accuracy. On the other hand, Cantera integrates the solution vector with the CVODES solver from SUNDIALS [69] with an adaptive time step.

A significant speed-up of 3 orders of magnitude is observed with React-DeepONet and LS-React-DeepONet for n-dodecane LTO compared to direct integration of the solutions in Cantera. Furthermore, LS-React-DeepONet performs approximately twice as fast as React-DeepONet or approximately 2500 times faster than a direct integration in Cantera.

A similar speed-up is not observed for H<sub>2</sub> combustion. The reaction system for H<sub>2</sub> combustion is a lot simpler and as a result, the Cantera solver does not take a long time to integrate the solution. Despite that, React-DeepONet for H<sub>2</sub> combustion is approximately 12 times faster.

It should be noted that the predictions from React-DeepONet are made on JAX [65] utilizing just-in-time compilation, which decreases the prediction time by an order of magnitude. The maximum integration time step is limited by the window size  $\Delta t$  over which the solution operator is learned. The window size can be further enlarged, which would translate to an even larger speed-up in each case.

#### 4. Conclusions

A combustion chemistry acceleration scheme is proposed and validated. It is based on DeepONets, which project solutions of thermochemical scalars' vectors over small and adaptive time increments. Instead of evolving a large solution vector of all the thermochemical scalars, a selected subset of representative thermochemical scalars is evolved. The remaining thermochemical scalars are reconstructed through ANNs. To enhance the model predictions' robustness and the efficiency of training, the proposed framework employs a novel training mechanism that is based on prescribed short-horizon time windows where intermediate solutions also act as initial states for

training. It also features an augmentation of the original DeepONet architecture with a parameter input sub-network named ParamNet. A latent space kinetics identification using an autoencoder is applied to improve computational efficiency further and broaden the model's applicability. The approach is validated to predict hydrogen-air and n-dodecane LTO. Validations of the approach show that it can predict the evolution of species and temperature over relevant integration times. Most importantly, the approach is enabling significant speedups in chemistry integration, which are more pronounced for more complex fuels.

The React-DeepONet framework with its promising results of low generalization error and computationally-efficient solution inference is a viable replacement for traditional ODE solvers in combustion simulations. Future extensions of the use of the proposed frameworks are related to their implementation in computational fluid dynamics (CFD) codes within the formulations of direct numerical simulations (DNS), large-eddy simulations (LES), and Reynolds-averaged Navier–Stokes (RANS) equations. In these simulations, chemistry integration represents a formidable bottleneck. Adding strategies to select representative scalars to recover all species in the chemical system will be investigated in conjunction with the development and implementation of the React-DeepONet and the LS-React-DeepONet. Another relevant effort is the design and selection of low-dimensional simulations that can approximate the composition space accessed in reacting CFD simulations.

#### CRediT authorship contribution statement

**Anuj Kumar:** Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Conceptualization.  
**Tarek Echekki:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Investigation, Conceptualization.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request.

## Acknowledgments

This research was funded by Aramco Americas, Saudi Arabia through a contract number ASC Agreement No. CW47993. The foundational code of the React-DeepONet framework has adopted some algorithms from the DeepONet [source code](#) developed by Prof. Perdikaris' group. We are grateful to our group member, Mr. Kevin Gitushi, for providing help in training data generation and representative set identification for n-dodecane oxidation.

## References

- [1] Pope S. Computationally efficient implementation of combustion chemistry using in situ adaptive tabulation. *Combust Sci Tech* 1997;1:41–63.
- [2] Toner S, Moriarty N, Frenklach M, Brown N. Computational economy improvements in PRISM. *Int J Chem Kinet* 2003;35:438–52.
- [3] Liang L, Stevens J, Raman S, Farrell J. The use of dynamic adaptive chemistry in combustion simulation of gasoline surrogate fuels. *Combust Flame* 2009;156:1493–502.
- [4] Continuo F, Jeanmart H, Lucchini T, D'Errico G. Coupling of in situ adaptive tabulation and dynamic adaptive chemistry: An effective method for solving combustion in engine simulations. *Proc Combust Inst* 2011;33:3057–64.
- [5] Sun W, Ju Y. A multi-timescale and correlated dynamic adaptive chemistry and transport (CO-DACT) method for computationally efficient modeling of jet fuel combustion with detailed chemistry and transport. *Combust Flame* 2017;184:297–311.
- [6] D'Alessio G, Cuoci A, Aversano G, Bracconi M, Stagni A, Parente A. Impact of the partitioning method on multidimensional adaptive-chemistry simulations. *Energies* 2020;13.
- [7] Maas U, Pope S. Simplifying chemical kinetics - intrinsic low-dimensional manifolds in composition space. *Combust Flame* 1992;88:239–64.
- [8] Lam S, Goussis D. The CSP method for simplifying kinetics. *Int J Chem Kinet* 1994;26:461–86.
- [9] Christo F, Masri A, Nebot E, Pope S. An integrated PDF/neural network approach for simulating turbulent reacting systems. In: 26th symp. (int.) on combustion. 1996, p. 43–8.
- [10] Christo F, Masri A, Nebot E. Artificial neural network implementation of chemistry with PDF simulation of H<sub>2</sub>/CO<sub>2</sub> flames. *Combust Flame* 1996;106:406–27.
- [11] Blasco J, Fueyo N, Dopazo C, Ballester J. Modelling the temporal evolution of a reduced combustion chemical system with an artificial neural network. *Combust Flame* 1998;113:38–52.
- [12] Blasco J, Fueyo N, Dopazo C, Chen J. A self-organizing-map approach to chemistry representation in combustion applications. *Combust Theory Model* 2000;4:61–76.
- [13] Chatzopoulos AK, Rigopoulos S. A chemistry tabulation approach via Rate-Controlled Constrained Equilibrium (RCCE) and Artificial Neural Networks (ANNs), with application to turbulent non-premixed CH<sub>4</sub>/H<sub>2</sub>/N<sub>2</sub> flames. *Proc Combust Inst* 2013;34:1465–73.
- [14] Franke LL, Chatzopoulos AK, Rigopoulos S. Tabulation of combustion chemistry via Artificial Neural Networks (ANNs): Methodology and application to LES-PDF simulation of Sydney flame L. *Combust Flame* 2017;185:245–60.
- [15] Ranade R, Alqahtani S, Farooq A, Echehki T. An ANN based hybrid chemistry framework for complex fuels. *Fuel* 2019;241:625–36.
- [16] Ranade R, Alqahtani S, Farooq A, Echehki T. An extended hybrid chemistry framework for complex hydrocarbon fuels. *Fuel* 2019;251:276–84.
- [17] Chen J, Blasco J, Fueyo N, Dopazo C. An economical strategy for storage of chemical kinetics: Fitting in situ adaptive tabulation with artificial neural networks. *Proc Combust Inst* 2000;28:115–21.
- [18] Wan K, Barnaud C, Vervisch L, Domingo P. Chemistry reduction using machine learning trained from non-premixed micro-mixing modeling: Application to DNS of a syngas turbulent oxy-flame with side-wall effects. *Combust Flame* 2020;220:119–29.
- [19] Wan K, Barnaud C, Vervisch L, Domingo P. Machine learning for detailed chemistry reduction in DNS of a syngas turbulent oxy-flame with side-wall effects. *Proc Combust Inst* 2021;38:2825–33.
- [20] Alqahtani S, Echehki T. A data-based hybrid model for complex fuel chemistry acceleration at high temperatures. *Combust Flame* 2021;223:142–52.
- [21] Readshaw T, Ding T, Rigopoulos S, Jones WP. Modeling of turbulent flames with the large eddy simulation-probability density function (LES-PDF) approach, stochastic fields, and artificial neural networks. *Phys Fluids* 2021;33.
- [22] Echehki T, Farooq A, Ihme M, Sarathy SM. Machine learning for combustion chemistry. In: Swaminathan N, Parente A, editors. Machine learning and its application to reacting flows: ML and combustion. Cham: Springer International Publishing; 2023, p. 117–47.
- [23] Owoyele O, Pal P. ChemNODE: A neural ordinary differential equations framework for efficient chemical kinetic solvers. *Energy AI* 2022;7.
- [24] Zhang T, Zhang Y, E W, Ju Y. DLODE: A deep learning-based ODE solver for chemical kinetics. In: AIAA SCITECH 2021 forum. no. AIAA Paper 2021-1139, 2021.
- [25] Kim S, Ji W, Deng S, Ma Y, Rackauckas C. Stiff neural ordinary differential equations. *Chaos* 2021;31.
- [26] Galassi R, Ciottoli P, Valorani M, Im H. An adaptive time-integration scheme for stiff chemistry based on computational singular perturbation and artificial neural networks. *J Comput Phys* 2022.
- [27] Lu L, Jin P, Pang G, Zhang Z, Karniadakis GE. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nat Mach Intell* 2021;3:218–29.
- [28] Chen T, Chen H. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE Trans Neural Netw* 1995;6:911–7.
- [29] Lin C, Li Z, Lu L, Cai S, Maxey M, Karniadakis GE. Operator learning for predicting multiscale bubble growth dynamics. *J Chem Phys* 2021;154:104118.
- [30] Lin C, Maxey M, Li Z, Karniadakis GE. A seamless multiscale operator neural network for inferring bubble dynamics. *J Fluid Mech* 2021;929:A18.
- [31] Cai S, Wang Z, Lu L, Zaki TA, Karniadakis GE. DeepM&Mnet: Inferring the electroconvection multiphysics fields based on operator approximation by neural networks. *J Comput Phys* 2021;436:110296.
- [32] Liu L, Cai W. DeepPropNet - A recursive deep propagator neural network for learning evolution PDE operators. 2022, ArXiv abs/2202.13429.
- [33] Di Leoni PC, Lu L, Meneveau C, Karniadakis GE, Zaki TA. Neural operator prediction of linear instability waves in high-speed boundary layers. *J Comput Phys* 2023;474:111793.
- [34] Wang S, Perdikaris P. Long-time integration of parametric evolution equations with physics-informed DeepONets. *J Comput Phys* 2023;475:111855.
- [35] Venturi S, Casey T. SVD perspectives for augmenting DeepONet flexibility and interpretability. *Comput Methods Appl Mech Engrg* 2023;403:115718.
- [36] Gitushi KM, Ranade R, Echehki T. Investigation of deep learning methods for efficient high-fidelity simulations in turbulent combustion. *Combust Flame* 2022;236.
- [37] Kumar A, Echehki T. A framework for combustion chemistry acceleration with DeepONets. 2023, arXiv:2304.12188.
- [38] Goswami S, Jagtap AD, Babae H, Susi BT, Karniadakis GE. Learning stiff chemical kinetics using extended deep neural operators. 2023, arXiv:2302.12645.
- [39] Kumar A, Echehki T. Physics - informed machine learning for reduced space chemical kinetics. In: NeurIPS 2023 workshop: Machine learning and the physical sciences, New Orleans, Dec 10-16, 2023. 2023.
- [40] Goswami S, Jagtap AD, Babae H, Susi BT, Karniadakis GE. Learning stiff chemical kinetics using extended deep neural operators. *Comput Methods Appl Mech Engrg* 2024;419:116674.
- [41] Hadorn PS. Shift-DeepONet: Extending deep operator networks for discontinuous output functions (Master's project), ETH Zurich; 2022, Computational Science and Engineering, A full MASTERSTHESIS entry.
- [42] Mao Z, Lu L, Marxen O, Zaki TA, Karniadakis GE. DeepM&Mnet for hypersonics: Predicting the coupled flow and finite-rate chemistry behind a normal shock using neural-network approximation of operators. *J Comput Phys* 2021;447.
- [43] Wang S, Wang H, Perdikaris P. Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. *Sci Adv* 2021;7:eabi8605.
- [44] Lanthaler S, Mishra S, Karniadakis GE. Error estimates for DeepONets: a deep learning framework in infinite dimensions. *Trans Math Appl* 2022;6.
- [45] Deng B, Shin Y, Lu L, Zhang Z, Karniadakis GE. Convergence rate of DeepONets for learning operators arising from advection-diffusion equations. 2021, ArXiv abs/2102.10621.
- [46] Pope S. Small scales, many species and the manifold challenges of turbulent combustion. *Proc Combust Inst* 2013;34:1–31.
- [47] Wang S, Teng Y, Perdikaris P. Understanding and mitigating gradient pathologies in physics-informed neural networks. *SIAM J Sci Comput* 2020;43:A3055–81.

- [48] Lu L, Meng X, Cai S, Mao Z, Goswami S, Zhang Z, Karniadakis GE. A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. *Comput Methods Appl Mech Engrg* 2022;393:114778.
- [49] Echehki T, Mirgolbabaie H. Principal component transport in turbulent combustion: A posteriori analysis. *Combust Flame* 2015;162:1919–33.
- [50] Owoyele O, Echehki T. Toward computationally efficient combustion DNS with complex fuels via principal component transport. *Combust Theory Model* 2017;21:770–98.
- [51] Mirgolbabaie H, Echehki T, Smaoui N. A nonlinear principal component analysis approach for turbulent combustion composition space. *Int J Hydrog Energy* 2014;39:4622–33.
- [52] Lu T, Law C. A directed relation graph method for mechanism reduction. *Proc Combust Inst* 2005;30:1333–41.
- [53] Gao X, Yang S, Sun W. A global pathway selection algorithm for the reduction of detailed chemical kinetic mechanisms. *Combust Flame* 2016;167:238–47.
- [54] Zdybał K, Sutherland JC, Parente A. Manifold-informed state vector subset for reduced-order modeling. *Proc Combust Inst* 2023;39:5145–54.
- [55] Oommen V, Shukla K, Goswami S, Dingreville R, Karniadakis G. Learning two-phase microstructure evolution using neural operators and autoencoder architectures. *NJP Comput Mater* 2022;8. <http://dx.doi.org/10.1038/s41524-022-00876-7>.
- [56] Dikeman H, Zhang H, Yang S. Stiffness-reduced neural ODE models for data-driven reduced-order modeling of combustion chemical kinetics. In: *AIAA SCITECH 2022 forum*. 2022, <http://dx.doi.org/10.2514/6.2022-0226>.
- [57] Champion K, Lusch B, Kutz JN, Brunton SL. Data-driven discovery of coordinates and governing equations. *Proc Natl Acad Sci* 2019;116:22445–51.
- [58] Lee K, Carlberg KT. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *J Comput Phys* 2020;404:108973.
- [59] Goodwin DG, Speth RL, Moffat HK, Weber BW. Cantera: An object-oriented software toolkit for chemical kinetics, thermodynamics, and transport processes. 2021, <https://www.cantera.org>, Version 2.5.1.
- [60] Almeldein A, Van Dam N. Accelerating chemical kinetics calculations with physics informed neural networks. In: *Internal combustion engine division fall technical conference*, vol. ASME 2022 ICE Forward Conference, 2022, <http://dx.doi.org/10.1115/ICEF2022-90371>, arXiv:<https://asmedigitalcollection.asme.org/ICEF/proceedings-pdf/ICEF2022/86540/V001T06A007/6950405/v001t06a007-icef2022-90371.pdf>.
- [61] Haghshenas M, Mitra P, Santo ND, Schmidt DP. Acceleration of chemical kinetics computation with the learned intelligent tabulation (LIT) method. *Energies* 2021;14(23). <http://dx.doi.org/10.3390/en14237851>, URL <https://www.mdpi.com/1996-1073/14/23/7851>.
- [62] Wang H, Xu R, Wang K, Bowman CT, Hanson RK, Davidson DF, Brezinsky K, Egolfopoulos FN. A physics-based approach to modeling real-fuel combustion chemistry - I. Evidence from experiments, and thermodynamic, chemical kinetic and statistical considerations. *Combust Flame* 2018;193:502–19.
- [63] Zádor J, Taatjes C, Fernandes R. Kinetics of elementary reactions in low-temperature autoignition chemistry. *Prog Energy Combust Sci* 2011;37:371–421.
- [64] Knio OM, Najm HN, Wyckoff PS. A semi-implicit numerical scheme for reacting flow: II. Stiff, operator-split formulation. *J Comput Phys* 1999;154:428–67.
- [65] Frostig R, Johnson M, Leary C. Compiling machine learning programs via high-level tracing. 2018, URL <https://mlsys.org/Conferences/doc/2018/146.pdf>.
- [66] Kingma D, Ba J. Adam: A method for stochastic optimization. In: *International conference on learning representations*. 2014.
- [67] Burke MP, Chaos M, Ju Y, Dryer FL, Klippenstein SJ. Comprehensive H<sub>2</sub>/O<sub>2</sub> kinetic model for high-pressure combustion. *Int J Chem Kinet* 2012;44:444–74.
- [68] Wang H, Dames E, Sirjean B, Sheen D, Tangko R, Violi A, Lai J, Egolfopoulos F, Davidson D, Hanson R, et al. A high-temperature chemical kinetic model of n-alkane (up to n-dodecane), cyclohexane, and methyl-, ethyl-, n-propyl and n-butyl-cyclohexane oxidation at high temperatures. 2010, p. 19, *JetSurF version 2*.
- [69] Hindmarsh AC, Brown PN, Grant KE, Lee SL, Serban R, Shumaker DE, Woodward CS. SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers. *ACM Trans Math Software (TOMS)* 2005;31:363–96.